Robotics and Autonomous Systems 195 (2026) 105163

Contents lists available at ScienceDirect

Robotics and Autonomous Systems

journal homepage: www.elsevier.com/locate/robot

Multi-Agent Reinforcement Learning for Zero-Shot Coverage Path Planning with Dynamic UAV Networks☆

José P. Carvalho *, A. Pedro Aguiar

Faculty of Engineering, University of Porto, Rua Dr. Roberto Frias, Porto, 4200-465, Portugal
SYSTEC-ARISE Research Center for System and Technologies, Rua Dr. Roberto Frias, Porto, 4200-465, Portugal

ARTICLE INFO

Keywords:
Multi-Agent Reinforcement Learning
Coverage path planning
Zero-shot generalization
Dynamic UAV network
Centralized Training Decentralized learning
Multi-robot systems

ABSTRACT

Recent advancements in autonomous systems have enabled the development of intelligent multi-robot systems for dynamic environments. Unmanned Aerial Vehicles play an important role in multi-robot applications such as precision agriculture, search-and-rescue, and wildfire monitoring, all of which rely on solving the coverage path planning problem. While Multi-Agent Coverage Path Planning approaches have shown potential, many existing methods lack the scalability and adaptability needed for diverse and dynamic scenarios. This paper presents a decentralized Multi-Agent Coverage Path Planning framework based on Multi-Agent Reinforcement Learning with parameter sharing and Centralized Training with Decentralized Execution. The framework incorporates a customized Rainbow Deep-Q Network, a size-invariant reward function, and a robustness and safety filter to ensure completeness and reliability in dynamic environments. Our training pipeline combines curriculum learning, domain randomization, and transfer learning, enabling the model to generalize to unseen scenarios. We demonstrate zero-shot generalization on scenarios with significantly larger maps, an increased number of obstacles, and a varying number of agents compared to what is seen during training. Furthermore, the models can also adapt to more structured maps and handle different tasks, such as search-and-rescue, without the need for retraining.

1. Introduction

Developments in intelligent autonomous systems have brought a shift in the automation paradigm. Autonomous systems and, more specifically, robots are no longer confined to simple, repetitive tasks in structured environments as they start being part of dynamic environments with complex decision-making processes. The next natural step for these robots (or agents) is to communicate and cooperate with each other, forming Multi-Robot Systems [1]. Having autonomous systems composed of more than one robot offers inherent advantages, including the ability to tackle more complex tasks, increase redundancy, and achieve greater overall efficiency compared to single-agent systems or uncooperative multi-agent frameworks [2].

Unnamed Aerial Vehicles (UAV) are at the forefront of the future of intelligent autonomous multi-robot systems, providing efficient aerial data collection while being flexible and cost-efficient. This makes them find multiple applications in precision agriculture [3], surveillance and monitoring [4], photogrammetry [5], search-and-rescue [6], infrastructure inspection [7], wildfire monitoring and prevention [8], and in the military sector [9]. Moreover, according to the 2022 JRC Technical Report [10], Europe experienced its largest burnt area since 2006, exceeding 8600 km^2, with more than 1000 km^2 being protected areas of Europe's Natura 2000 network, causing long-lasting effects on Europe's biodiversity reservoirs. Among the proposed mitigation measures is the "development of early warning and information systems for wildfires", where UAV-based monitoring of critical areas could contribute to this effort.

Most of the mentioned UAV applications are related to the problem of coverage path planning (CPP). CPP consists of the computation of collision-free paths that cover all points of an area or volume of interest. Due to its generality, it is a long-standing task in robotics, with multiple proposed solutions over the years. While most focus on the single-agent approach to the problem, there has been a growing interest in utilizing multiple robots for coverage tasks or Multi-Agent CPP (mCPP) in recent years. The first approaches to mCPP [11] were mostly based on heuristics and meta-heuristics [12], graph search [13], or focused on partitioning the area that each agent will tackle [14]. In general, these approaches lack the flexibility to handle dynamic environments, limited communication, and mCPP tasks with other complex objectives.

☆ This article is part of a Special issue entitled: 'Planning & Learning' published in Robotics and Autonomous Systems.

* Corresponding author at: Faculty of Engineering, University of Porto, Rua Dr. Roberto Frias, Porto, 4200-465, Portugal.
E-mail addresses: jpcarvalho@fe.up.pt (J.P. Carvalho), pedro.aguiar@fe.up.pt (A.P. Aguiar).

https://doi.org/10.1016/j.robot.2025.105163

Available online 26 September 2025
0921-8890/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

To tackle these gaps, the research community has steered into Reinforcement Learning (RL) approaches, which have shown the potential to solve demanding decision-making problems, even in Multi-Agent Scenarios [15]. While Multi-Agent Reinforcement Learning (MARL) has been successfully applied to mCPP, the current approaches in the literature are usually focused on a sub-problem of mCPP [16], or restricted to single scenarios [17], or the designed architectures do not scale with the number of agents [18], and other issues that limit their "out-of-the-box" usability.

In this paper, we address the mCPP problem from a zero-shot generalization perspective. The goal is to develop a MARL framework that can perform mCPP in scenarios that were never seen during training. We envision it used as a Dynamic UAV network that can be rapidly deployed anywhere without the need for retraining. This flexibility is particularly valuable in scenarios such as disaster relief, which are often unpredictable and require urgent action. These scenarios can have different numbers of agents, map sizes, obstacles, and starting positions. Furthermore, they are dynamic, as the framework must cope with the addition or removal of agents and obstacles during deployment.

To this end, we propose a decentralized mCPP framework based on MARL with parameter sharing [19] and Centralized Training Decentralized Execution (CTDE) [20,21]. This approach allows agents to share experiences and collaboratively learn a singular policy during training, which accelerates learning and mitigates the effects of non-stationarity [22], while maintaining scalability and flexibility during execution. Within the CTDE architecture, we design a tailored version of the Rainbow Deep Q-Network (DQN) [23] and Value Decomposition Networks (VDN) for mCPP. The problem is formulated as a Stochastic Markov Game, adapted to centralized training by using a shared observation space that is centered on the ego agent. The observation space is further designed to adapt to any map size, utilizing local information while compressing the remaining global information. The generalization effort is facilitated by introducing a novel reward function that makes the value function size invariant and lets the cooperation level between agents be manually tuned. In addition, we guarantee the safety and completeness of our framework by introducing a safety and robustness filter [24]. Finally, we define a training framework based on domain randomization, curriculum learning, and transfer learning, enabling the UAV network to systematically progress from simpler tasks to more complex ones.

We conduct extensive hyperparameter tuning on the reward functions and curriculum learning parameters, alongside ablation studies, to justify and explain the design choices that were empirically shown to achieve better results, including an in-depth comparison between the adapted Rainbow DQN and VDN. The framework's ability to perform zero-shot generalization is demonstrated through Monte Carlo simulations in scenarios that are independent and identically distributed (i.i.d.) relative to the training set. We also test in scenarios that are out of distribution, first by generating larger maps with more obstacles, and in a second instance by testing on structured maps and changing the task to search-and-rescue.

The contributions of our work are summarized as follows:

- We address the multi-agent Coverage Path Planning (mCPP) problem from a zero-shot generalization perspective by formulating it as a Partially Observable Stochastic Game (POSG) and solving it using Multi-Agent Reinforcement Learning (MARL).
- We propose and validate a novel reward function that enables tunable levels of cooperation and induces size-invariant value functions, directly supporting generalization across different environment sizes and numbers of agents.
- We develop a learning framework based on value-based methods within a Centralized Training and Decentralized Execution (CTDE) paradigm. To effectively address the challenges of Zero-Shot Generalization (ZSG) and the multi-agent setting, we introduce a custom training pipeline that integrates curriculum learning, domain randomization, transfer learning, and learning from demonstrations. Our ablation studies demonstrate that these components are critical for achieving strong generalization and near-optimal performance.
- We design a decentralized robustness and safety filter, which formally ensures safety and completeness in the mCPP problem.
- We conduct an in-depth analysis of multiple design choices for achieving zero-shot generalization with parameter sharing, Deep-Q Networks, and Value Decomposition Networks, and examine the trade-offs between these methods.
- Through extensive testing, we demonstrate that models trained by our framework have strong zero-shot capabilities, by handling scenarios with up to twice the number of obstacles, environments that are 2.77 times larger, have five times more agents than in the training set, as well as structured scenarios that are significantly different from those encountered during training.

The remainder of this paper is organized as follows: Section 2 presents a short review of the literature on RL approaches to single and multi-agent CPP. Section 3 describes the problem of Multi-Agent CPP, defines the agent and environment interactions and constraints, and formulates CPP through the lens of an optimization problem. The formulation is expanded on Section 4, where the optimization problem is transformed into a Stochastic Markov Game, defining the Observation Space, Reward Structures, and Action-Space shaping techniques. The Multi-Agent Reinforcement Learning methods used to solve the Stochastic Game are presented in Section 5, and its results are discussed in Section 6. Finally, the main conclusions and future research opportunities are highlighted in Section 7.

Notation

In this paper, the symbol $\mathbb{B}$ will denote the Boolean Domain. For variables, lower-case letters x will be considered scalars, upper-case letters X are matrices or vectors, and bold upper-case $\mathbf{X}$ will denote sets. The notation $\vec{\mathbf{0}}$ denotes a vector where all elements are equal to 0, and $\vec{\mathbf{1}}$ is a vector where all elements are equal to 1, whereas $\mathbb{1}(\cdot)$ is the indicator function. The operator $|\cdot|$ is the l_0 Norm, whereas $\|\cdot\|_p$ denotes the L_p Norms. The notation $\sim \mathbf{X}$ is used whenever a sampling operation is done to a set $\mathbf{X}$, the operator $\mathbb{E}$ denotes the expected value of a random variable, and $\mathbf{X}_1 \setminus \mathbf{X}_2$ represents the relative complement of $\mathbf{X}_2$ in $\mathbf{X}_1$ (also termed the set difference of $\mathbf{X}_1$ and $\mathbf{X}_2$). Considering a matrix $X \in \mathbb{B}^{n\times n}$, $X[x, y]$ denotes the value contained in row $x \in \mathbb{N}_0$ and column $y \in \mathbb{N}_0$; $X[x, :]$ denotes the whole vector contained in the row x and $X[x, i : o]$ denotes the vector in row x from indexes i to o where $i, o \in \mathbb{N}_0, i < o$. A summary of the notation can be seen in Table 1.

2. Related work

In this section, we focus primarily on Reinforcement Learning approaches to Multi-Agent Coverage Path Planning. Nonetheless, due to the scarcity of work, adjacent tasks such as patrolling, informative path planning, and data harvesting will also be considered as long as they directly involve the cooperation of multiple robots in tasks similar to CPP.

Coverage Path Planning is a common task in robotic applications that, in the most general form, consists of the computation of paths that visit all points of interest of a determined area or volume [25]. Due to the myriad of tasks that are similar or derivative of the CPP task, the topic is well-established in the literature, receiving multiple breakthroughs over the years. In one of the first surveys on the topic, [11] establishes the main concepts of the classical CPP task, introducing both heuristic and complete methods for the single-agent task. Interestingly, despite still being in the initial phase, the Multi-Robot problem is already addressed in this survey, being that solutions are mostly based on coordination via pheromone-aided online-graph search [26], and exact

Table 1
Summary of notation.

Symbol	Description
ξ	Set of Map Cells
$\mathbb{C}$	Set of Cells of Interest
$\mathbb{O}$	Set of Obstacles
Λ	Set of Non-Obstacle Cells
$\mathbb{J}$	Set of UAVs
$\mathbb{P}$	Set of UAV positions
$\mathbb{A}$	Action Space
$\mathbf{B}$	Set of UAV battery levels
$\mathbb{S}$	State Space
$\mathbf{P}$	State transition probability function
R	Reward Function
Ω	Observation Space
$\mathcal{O}$	Observation Function
M	Map Representation
E	Edge of the Map
Ψ	Compressed Map Information
$\mathcal{M}$	Compressed and Centered Map Representation
L_a	Vector of Last Taken Actions
e	Event
K	Hyperparameter that regulates cooperation levels
$\mathbb{H}$	Information State Space
π	Decision Making Policy
Δ	Filter Monitor
ϕ	Filter Intervention Scheme
θ	Neural Network Learning Parameters
λ_J	Time Save Factor
O_J	Overlap

cell decomposition [27]. More recent surveys [25,28] showcase the development of classical CPP algorithms, both in single and multiple-agent scenarios. Nonetheless, the field has seen a significant surge of interest with the rise of Deep Reinforcement Learning (DRL) and other machine learning-based strategies, which, while being deemed still immature [29], some authors [30] view their widespread use in the future use as "almost inevitable".

Reinforcement Learning approaches to mCPP can be divided into two groups: Tabular and Deep RL. The tabular approaches are usually the simplest ones, relying on a tabular representation of the Q-Function instead of using Neural Networks as function approximators. In practice, tabular methods are less complex, can come with convergence guarantees [31], and are more tractable and interpretable than the Deep RL counterpart. However, these methods suffer from the Curse of Dimensionality [32], as they require storing and exploring all possible combinations of states and actions. This makes them impractical or even unfeasible for the case of Zero-Shot Generalizability, where agents must be able to handle scenarios that were never explored during training. Nevertheless, these methods have been applied successfully to mCPP.

In [33], the authors tackle mCPP with Tabular Q-Learning and SARSA and propose a method of centralized Q-Learning. Due to the tabular nature and the centralized architecture, the method is limited to two agents and two small maps (5×5) with a single obstacle. On the other hand, [34] propose a Tabular distributed Q-Learning method named Q-Transversal, which introduces relevant ideas to the field, such as having an information map γ that is shared between all agents, the usage of Action-Space Shaping to reduce the size of the action-space whenever possible, combined with a heuristic policy to aid training process. The combination of these components aids in avoiding some issues of the curse of dimensionality, but the tabular nature is still limiting when it comes to zero-shot generalizability. Another example of Tabular Q-Learning combined with other methods that reduce the complexity of the RL decision-making problem can be seen in [35], where Dynamic Radar Networks (DRNs), composed of UAVs, are utilized for joint target detection and mapping.

Deep Reinforcement Learning (DRL) methods can potentially overcome the curse of dimensionality and other shortcomings of the tabular methods. They are very popular in decision-making problems with large state spaces, making them the most common option for mCPP tasks. The series of works [36–38] is among the most influential, developing core ideas for DRL approaches to the single-agent CPP task. These studies utilized the Deep-Q Network (DQN) algorithm, where the input is an image of the map representation; this map representation is centered on the agent's position and combines global and local map information at different resolutions to enhance performance. These innovations are generalized to the multi-agent setting in [39], where the authors tackle the problem of data harvesting with a group of UAVs. The results show that the DQN with CTDE can generalize to different numbers of agents and devices emitting data. However, the trained model is fixed to the map, meaning that zero-shot generalization is impossible for maps of different sizes.

The DQN and its variants are among the most common choices for CPP and mCPP. The Double Q-Network (DDQN) with CTDE is used to coordinate a cluster of 4 UAVs in an mCPP task [40]. The results are limited to a single obstacle-less scenario but show that the method can adapt to different agent starting positions. In [41], the authors conduct a study on the usage of the DDQN for mCPP in the context of Precision Agriculture. A centralized and independent version of the DDQN is implemented and compared, showing that centralizing the model results in significantly faster training and slightly better results. Nonetheless, centralization (without distributed execution) means that the number of UAVs is bound to the network architecture, making the model a singular point of failure and making it more difficult to deploy in real scenarios. A distributed approach to the DQN is presented in [42], where a group of UAVs must perform surveillance in an area by finding targets. One interesting property of the proposed model is that it was shown to be able to few-shot learn a previously unknown map. The distributed DQN is also used in [18], where the problem is formulated as continuous patrolling. It exploits graph structures to construct the MDP, being that the used maps are represented by graphs where the nodes have at most four neighbors. The state space is composed of the agent's local information, as in the neighboring nodes from its current position. The authors also propose a state-space representation where agents share their local information with each other; however, it has the shortcoming that it limits the trained model to a fixed number of agents. Furthermore, while the method shows some zero-shot generalizability capabilities, only two maps are addressed in this study. The series of works [17,43,44] studies the use-case of DQN-based methods for coverage, patrolling, monitoring, and informative path planning of Lake Ypacaraí. The results show that the methods can outperform other classical algorithms, but as they are specialized in the Lake Ypacaraíscenario, there are no results on their zero-shot generalizability.

In the same vein as the DQN, studies have been conducted on the adaptation of actor–critic single-agent algorithms to CPP. The Proximal Policy Optimization (PPO) [45] method is utilized for mCPP in 3D environments with UAVs in [46]. There is some emphasis on zero-shot generalization, as the agents are trained in randomly generated environments. Still, the study is limited to a single map, where the parameters that change are mostly the zones of interest, agents' starting position, and battery level. As for works with heterogeneous agents, [47] proposes a method based on Asynchronous Advantage actor–critic (A3C). It introduces novelties in the method by using mask attention and soft attention and incorporating LSTM layers. The study focuses on the typical objective of minimizing total coverage time, but with the constraint that any node cannot be visited more than three times. The results show that the developed method achieves more efficient coverage than the other seven baseline methods. However, the results are limited to only three maps with varying shapes.

On a slightly different type of task, in [48], the authors develop a MARL method based on Graph-Neural Networks, developing a new actor–critic method G-SAC. The method is shown to adapt to two

tasks, including centroid flocking and landmark covering.[1] The model displays zero-shot generalizability abilities, and the distributed architecture can handle large-scale multi-robot coordination. Another problem adjacent to CPP is Informative Path Planning. In [16], the authors use a team of UAVs to cooperatively gather sensor data, in their case, the temperature in a determined area of interest. Unlike the CPP approach, here the goal is to maximize an information-theoretic target (minimum-entropy), such that the agents must plan both the path in the 2D plane and control their height as it affects the accuracy and the range of the sensor reading. While not specifically addressing the generalization problem, the authors show that the method overcomes the Sim2Real gap by conducting real-life experiments. Finally, in [49], the authors propose a DRL method for UAV path planning in cellular-connected networks, aiming to minimize both flight time and communication outages. It introduces a quantum-inspired experience replay (QiER) mechanism to improve learning efficiency. The goal of the approach is to find efficient trajectories while minimizing cellular connection outage, even without full knowledge of the wireless environment.

Despite the existence of several RL approaches to mCPP, most are tailored for specific scenarios and do not take into account completeness and safety guarantees. While this can be reasonable whenever the model is trained for a specific scenario, and thus robustness can be inferred by conducting Monte-Carlo simulations, this is not adequate as an out-of-the-box solution that can adapt to most scenarios. In this paper, we focus on dealing with this gap in the literature by emphasizing Zero-Shot Generalizability [50]. To this end, we start by formulating the problem in a generic optimization framework and convert it into a Stochastic Partial Observable Stochastic Game. The combination of this formulation with a tailored training pipeline, a novel reward structure, and a safety and robustness filter enables our model to achieve zero-shot generalization abilities in mCPP tasks.

3. Problem statement

The following section introduces our formulation of the Multi-Agent Coverage Path Planning problem. It explores the environment and UAV (agent) models, including kinematic and energy constraints, inter-agent, and agent-environment interaction. After defining the system, we will formulate the coverage task as a minimum time problem and analyze it through an optimization framework.

3.1. Environment model

To keep the formulation as generic as possible, we consider an environment represented by a 2D square-shaped map ξ of size $m \times m$, where the set of all cell coordinates is defined as:

$$\xi = \{(x, y) \mid x, y \in \mathbb{N}_0,\ x, y < m\}. \tag{1}$$

The map is obtained via approximate cellular decomposition, and the cell size is equal to the agent size. We will also assume that any obstacles are static and cell-sized. In our framework, the obstacles can be randomly generated, meaning their number and position will vary depending on the scenario. Furthermore, while we consider the map to be square-shaped, and any other shape can be obtained by encoding out-of-bounds cells as obstacles.

Therefore, the map can be described as a 2D Matrix with two Boolean channels $C, O \in \mathbb{B}^{m \times m}$, corresponding to Cells of Interest and Obstacles, respectively. In addition, we define the set that contains the coordinates of all Cells of Interest as

$$\mathbb{C} = \{(x, y) \mid C[x, y] = 1, (x, y) \in \xi\}, \tag{2}$$

and the set $\mathbb{O}$, which contains the coordinates of all Obstacles, is defined as:

$$\mathbb{O} = \{(x, y) \mid O[x, y] = 1, (x, y) \in \xi\}. \tag{3}$$

Since the obstacles are randomly generated, it might happen that some cells of interest are not globally reachable. This issue would make some instances of the environment unsolvable, and therefore, if a cell $c \in \mathbb{C}$ meets this condition, it will be transferred to the set of obstacles $c' \in \mathbb{O}$ before the task starts. Also, note that a cell cannot be an obstacle and cell of interest simultaneously $\mathbb{C} \cap \mathbb{O} = \emptyset$, such that we define $\Lambda = \xi \setminus \mathbb{O}$ as the set that contains all non-obstacle-cells.

Definition 1. A cell c_f is said to be **globally reachable** if $\forall c_0 \in \Lambda$ there exists a path $\Phi = \{c_0, \dots, c_f\} \subset \Lambda$ such that $\forall k \in \{1, \dots, f\}$, the condition $\|c_k - c_{k-1}\|_1 = 1$ holds.

An example of a map can be visualized in Fig. 1.

3.2. UAV model

We will consider that for any scenario presented in this work, there will be a set $\mathbb{J}$ of $J \in \mathbb{N}$ deployed UAV. The j−th UAV occupies a position $P^j = (p_x^j, p_y^j) \in \Lambda$, which can be any non-obstacle-cells. In addition, we define the set that contains all UAVs' positions as:

$$\mathbb{P} = \{P^j \mid j = 1, \dots, J, P^j \in \Lambda\}. \tag{4}$$

The agents can update their position by taking an action $a_t^j \in \mathbb{A}^j$ at every discretized time-step t, where $\mathbb{A}^j$ is the Action Space. We will assume that the UAVs hover at a fixed altitude thus the movement is limited to xy-plane, resulting in the Action Space being described as:

$$\mathbb{A}^j = [North, South, West, East, Wait]. \tag{5}$$

The Action Space is homogeneous across all agents, such that $\mathbb{A}^1 = \mathbb{A}^2 = \cdots = \mathbb{A}^J$. Moreover, the global reference system is defined with the origin at the most northwestern point, resulting in the position of the jth evolving as:

$$P_{t+1}^j(a) = \begin{cases} P_t^j + (-1, 0) & \text{if } a_t^j = \text{North} \\ P_t^j + (1, 0) & \text{if } a_t^j = \text{South} \\ P_t^j + (0, -1) & \text{if } a_t^j = \text{West} \\ P_t^j + (0, 1) & \text{if } a_t^j = \text{East} \\ P_t^j + (0, 0) & \text{if } a_t^j = \text{Wait} \end{cases} \tag{6}$$

The problem has an energy constraint represented by the battery level of each UAV b^j, which is equal to the number of actions that the agent j can make. The set that contains all agents' battery levels will be referred to as $\mathbf{B}_t = \{b_t^1, \dots, b_t^J\}$. In our setup, this constraint is not too restrictive as the battery level will always be enough to fully cover the map by at least a five times factor $b_0^j > 5m^2$, independently of the number of agents. The primary purpose of this constraint is to make the problem finite, regardless of the used policy. The battery discharges according to the following equation:

$$b_{t+1}^j = b_t^j - 1 \tag{7}$$

3.3. Optimization problem

Considering that the agents can start at any non-obstacle cell and there is no specific position to end the task, the multi-agent coverage path planning problem can be seen as minimizing the number of discrete timesteps taken to visit the set of all cells of interest $\mathbb{C}$. Thus, we consider that the time evolution of the set of cells of interest can be described as:

$$\mathbb{C}_{t+1} = \mathbb{C}_t \setminus \mathbb{P}_t. \tag{8}$$

Intuitively, this equation translates that if the position of an agent P_t^j is at a cell of interest $c \in \mathbb{C}_t$, then at the next time-step, the cell c

[1] Different from CPP: "In cooperative coverage environment, the multi-robot system should cover N landmarks simultaneously" [48].

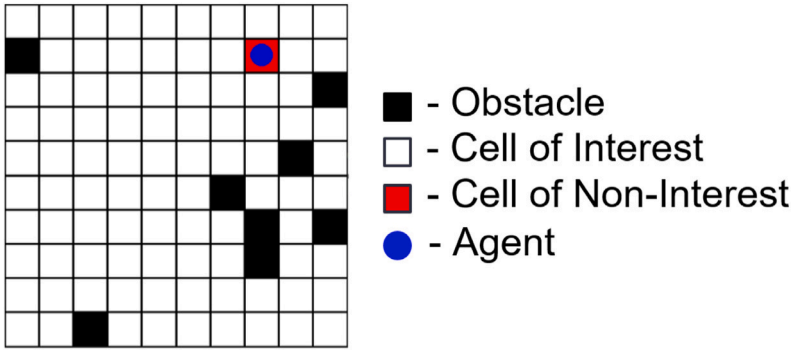


Fig. 1. Example of a map representation and the respective color code. The black color corresponds to all cells in $\mathbf{O}$, whereas white represents the cells of interest $\mathbf{C}$, and finally, the red cells are the ones that are not contained in any set $\neg\mathbf{C} \cap \neg\mathbf{O}$. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

is removed from the set of Cells of Interest, i.e., it was covered. In addition, the evolution of Eq. (8) is a strictly monotonic non-increasing function, resulting in the initial condition $|\mathbb{C}_0|$ being the maximum number of cells of interest possible, and problems, where more cells are added, are not considered during training.

Therefore, the generic optimization problem can be formulated as:

$$\begin{aligned} &\text{Minimize} \quad T \\ &\textbf{s.t.} \\ &(6) \wedge (7) \wedge (8) \\ &a_t^j \in \mathbb{A}^j \wedge P_t^j \in \boldsymbol{\Lambda} \wedge b_t^j > 0, \quad \forall t \in [0,T], \forall j \in \mathbb{J} \\ &i \neq j \implies P_t^i \neq P_t^j, \quad \forall t \in [0,T], \forall i,j \in \mathbb{J} \\ &\mathbb{C}_T = \emptyset. \end{aligned} \tag{9}$$

This formulation enforces that the goal of the problem is to minimize the terminal timestep T. This timestep is related to the coverage path planning problem through the terminal constraint of visiting all cells of interest, that is, the set of cells of interest $\mathbb{C}_t$ at time $t = T$ must be $\mathbb{C}_T = \emptyset$. In addition, we also enforce that the agents must not collide with the environment or other agents and that the agents must not run out of battery.

The generality of this formulation enables the optimization to be agnostic to the number of agents (either single or multi-agent), and the initial conditions $\mathbb{C}_0, \mathbb{O}_0, \mathbb{P}_0$ can be any as long the constraints are met, and all cells of interest are globally reachable.

4. Partially observable stochastic game framework

The proposed multi-agent optimization problem in Eq. (9) is generally intractable and cannot reliably be solved with classical optimization methods. To deal with this issue, we will employ a Multi-Agent Reinforcement Learning Framework. This section will address the adaptation of the presented optimization problem to a Partially Observable Stochastic Game (POSG), which can be defined through the tuple $(\mathbb{J}, \mathbb{S}, \mathbb{A}_\times, \mathbf{P}, \mathbf{R}_\times, \Omega_\times, \mathcal{O}, \gamma)$ [51], a multi-agent generalization of a Partially Observable Markov Decision Process (POMDP). Here, $\mathbb{J}$ is the set of J agents as previously defined, $\mathbb{S}$ is a finite set that contains all possible states s of the environment, and $\mathbb{A}_\times = \mathbb{A}^1 \times \cdots \times \mathbb{A}^J$ is the joint Action Space; $\mathbf{P} : \mathbb{S} \times \mathbb{A}_\times \to \mathbb{S}$ is the state transition probability function, $\mathbf{R}_\times : \mathbb{S} \times \mathbb{A}_\times \times \mathbb{S} \to \mathbb{R}^J$ denotes the joint reward function, defined as the collection $\mathbf{R}_\times = [R^1, \ldots, R^J]$ of individual reward functions, where each R^j maps a state transition to a scalar reward received by agent j. That is, $R^j : \mathbb{S} \times \mathbb{A}^j \times \mathbb{S} \to \mathbb{R}$. The joint observation space is defined through the finite set $\Omega_\times = \Omega^1 \times \cdots \times \Omega^J$ and $\mathcal{O} : \mathbb{S} \to \Omega$ is the observation function mapping states s to one agent's individual observations o^j. It is worth noting that the observation function $\mathcal{O}$ is modeled as a simplified approach to POMDPs, where the observation solely depends on the current state and the agent's observation mechanisms instead of a probability function $P_r(o^j \mid a^j, s')$[52].

Moreover, in our methodology, we will also consider Decentralized POMDPs (Dec-POMDP), which is a special case of the POSG where there exists a singular shared (team) reward signal, such that $\mathbf{R} : \mathbb{S} \times \mathbb{A}_\times \times \mathbb{S} \to \mathbb{R}$.

This framework is the one that best fits our coverage path planning problem, as agents do not communicate directly with each other. They are only aware of their own actions and rewards, and act based solely on their own observations. In addition, the agents might not be able to visualize the whole state, making this aspect also partially observable. In the following subsections, we will describe the design process of the Observation Space, Safety, and Robustness Filter, as well as propose multiple Reward Structures.

4.1. Observation space

Consider the problem formulation presented in Section 3. In a full information scenario, the problem can be transformed into the Markovian setting by representing the state as the tuple:

$$s(t) = (M(t), \mathbf{B}_t), \tag{10}$$

where $\mathbf{B}_t$ is the set that contains all the agents' battery level and $M(t) \in \mathbb{B}^{4 \times m \times m}$, being that each channel corresponds to:

- **Channel 1:** Cells of Interest $\mathbb{C}_t$
- **Channel 2:** Cells of Non-Interest $\neg\mathbb{C}_t \cap \neg\mathbb{O}_t$
- **Channel 3:** Obstacles $\mathbb{O}_t$
- **Channel 4:** Agents' Positions $\mathbb{P}_t$

It is important to note that the second channel could be omitted as $\mathbb{C}_t \cap \mathbb{O}_t = \emptyset$, and thus the information would be implicitly encoded without the channel. However, empirically, explicitly representing these cells that had already been visited improved agent performance. Another design choice was to include the fourth channel, which represents the agents's position. While it would be possible to use a tuple of positions instead of using this much larger representation, in the context of multi-agent reinforcement learning, this representation scales better, as its size does not increase with the number of agents J, unlike the tuple. In addition, it enables Map Centering [38] and its usage as Agent Indicator [19].

4.1.1. Map centering

The first step of creating the jth agent observation is to center the map $M(t)$ on the agent's position. Theile et al. [38] introduced this concept, and it was shown to empirically improve performance in the context of single-agent coverage. In this scenario, the main improvement is due to the neural network only needing to learn spatial relationships on the agent's reference frame instead of the global frame.

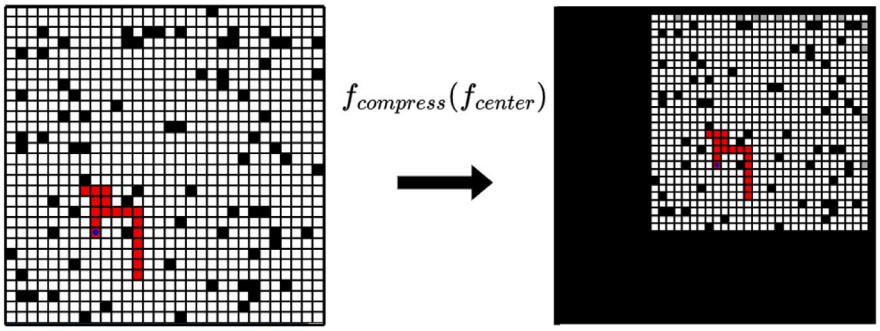


Fig. 2. Example of the map centering and compression on a 30 × 30 Map. In the compressed map, the top and right edges result from averaging edges that were cropped from the original map.

In addition to this relevant improvement, in the case of the multi-agent scenario, centering the map also serves as an Agent Indicator of the ego-agent, as the agent will always be at the center of the map representation. Finally, since the agents are all represented in the same manner, and both the goals and action spaces are similar, this representation can aid the learning model in reasoning what the other agents are most likely to do without any explicit communication.

Thus, consider the global map representation given by the Matrix $M \in \mathbb{R}^{4\times m\times m}$, one can apply the centering function to obtain the map $M_c^j \in \mathbb{R}^{4\times m_c\times m_c}$ centered on the agent j position. We define this function as a mapping

$$f_{center}(M, P^j, X) : \mathbb{R}^{4\times m\times m} \times \boldsymbol{\Lambda} \times \mathbb{R}^4 \rightarrow \mathbb{R}^{4\times m_c\times m_c} \tag{11}$$

that takes into account the agent position P^j and the padding vector $X = (0, 0, 1, 0)^T$, wherein all padded cells are encoded as obstacles (thus the value of 1 in the third channel of the vector). Therefore, the elements of M_c^j can be computed as [38]

$$M_c^j[x, y] = \begin{cases} M^j[x + p_x^j - m + 1, y + p_y^j - m + 1] & \text{if } (m \leq x + p_x^j + 1 < 2\ m) \\ \quad \wedge\ (m \leq y + p_y^j + 1 < 2\ m) & \\ X & \text{otherwise.} \end{cases} \tag{12}$$

Note that in the original work, this formulation is limited by the maximum possible value of m_c, as it cannot be variable due to the neural network architecture. In our setup, we will consider that the maximum map size $m' = 20$, which corresponds to $m'_c = 39$. As such, all maps smaller than 20 × 20 are simply padded with obstacles by simply using the centering function. However, due to our focus on generalization, we will extend this technique by also considering maps larger than this limit that would otherwise need to be cropped. We will address this problem by compressing information on the edges of the representation [53].

4.1.2. Map compression

Consider the Map Representation M_c, which was already processed by the centering function f_{center}. Let its size m_c be unbounded, while the neural network can only process maps up to m'_c. To deal with the issue where $m_c > m'_c$, we use a compression scheme where edges that extend beyond m'_c are compressed by averaging their values and merging them into the boundary edge within m'_c, thereby retaining part of the global information while maintaining the representation within the network's size limit. Thus, from the centered map $M_c \in \mathbb{R}^{4\times m_c\times m_c}$, consider the visible submatrix $M'_c \in \mathbb{R}^{4\times m'_c\times m'_c}$. Furthermore, in the original centered map, let the matrix $E_d \in \mathbb{R}^{4\times m_c\times n}$ contain the n edges that extend beyond m'_c in one of the four possible directions $d \in \{N, S, W, E\}$.

Inside this matrix of edges, there is a submatrix $E'_d \in \mathbb{R}^{4\times m'_c\times n}$, which has the part of the cropped edge that is visible in the non-cropped dimension. We define this submatrix as

$$E'_d = E_d[:, i_s : i_e, :], \tag{13}$$

where $i_s, i_e \in \mathbb{N}_0$ are the start and end index respectively. While this would be enough if the edge is only cropped in one dimension, it is possible that the interval between i_s and i_e does not contain the whole non-padded edge, i.e the edge was cropped in more than one direction. Therefore, consider the offsets $o_s, o_e \in \mathbb{N}_0$, so that the submatrix obtained by $E_d[:, i_s - o_s : i_e + o_e, :]$ contains the whole edge. Therefore, the compressed edge $E_d^C \in \mathbb{R}^{4\times m'_c}$ can be obtained by using the compress function

$$E_d^C = f_{compress}(E_d, i_s, i_e, o_s, o_e), \tag{14}$$

and the function $f_{compress}$ can be seen as the mapping

$$f_{compress} : \mathbb{R}^{4\times m_c\times n} \times \mathbb{N}_0^4 \rightarrow \mathbb{R}^{4\times m'_c}, \tag{15}$$

where the individual elements of the compressed edge E_d^C can be computed as

$$E_d^C[l, k] = \begin{cases} \sum_{x=i_s-o_s}^{i_s} \sum_{y=0}^{n} \frac{E_d[l,x,y]}{n\cdot o_s} & \text{if } k = 0 \wedge o_s > 0 \\ \sum_{x=i_e}^{i_e+o_e} \sum_{y=0}^{n} \frac{E_d[l,x,y]}{n\cdot o_e} & \text{if } k = m'_c \wedge o_e > 0 \\ \sum_{y=0}^{n} \frac{E_d[l,k,y]}{n} & \text{otherwise.} \end{cases} \tag{16}$$

Finally, after centering and compressing the map representation, we concatenate a new edge in every direction. This edge consists exclusively of obstacles, and its objective is visual consistency, meaning that the outermost edge of the map is a wall independently of map size and agent position. Thus, we obtain our final map representation $\mathcal{M}^j \in \mathbb{R}^{4\times(m'_c+2)\times(m'_c+2)}$. This representation enables the agent to distinguish himself from others and to comprehend maps larger than its immediate visual frame. To further empower this representation, the observation space incorporates an additional tensor denoted as $\Psi^j \in \mathbb{R}^{4\times 2}$.

$$\Psi^j = \begin{bmatrix} \epsilon_N & \rho_N \\ \epsilon_S & \rho_S \\ \epsilon_W & \rho_W \\ \epsilon_E & \rho_E \end{bmatrix}. \tag{17}$$

In this tensor, each of the four directions is associated with two values: the count of compressed edges ϵ_d and the number of points of interest ρ_d in the corresponding direction d. An example of centering and compressing the map can be visualized in Fig. 2.

4.1.3. Path awareness and state looping

Our previous work [53] explores the concept of state-looping and how it negatively impacts the generalization and robustness of Deep Reinforcement Learning for POMDPs and specifically for the single-agent coverage path planning task. These loops are introduced by uncertainties and errors in the function approximation, leading to a sequence of actions given by a deterministic policy that does not induce change in the sequence of observations. As the observations do not change, the policy remains unchanged, resulting in an infinite loop.

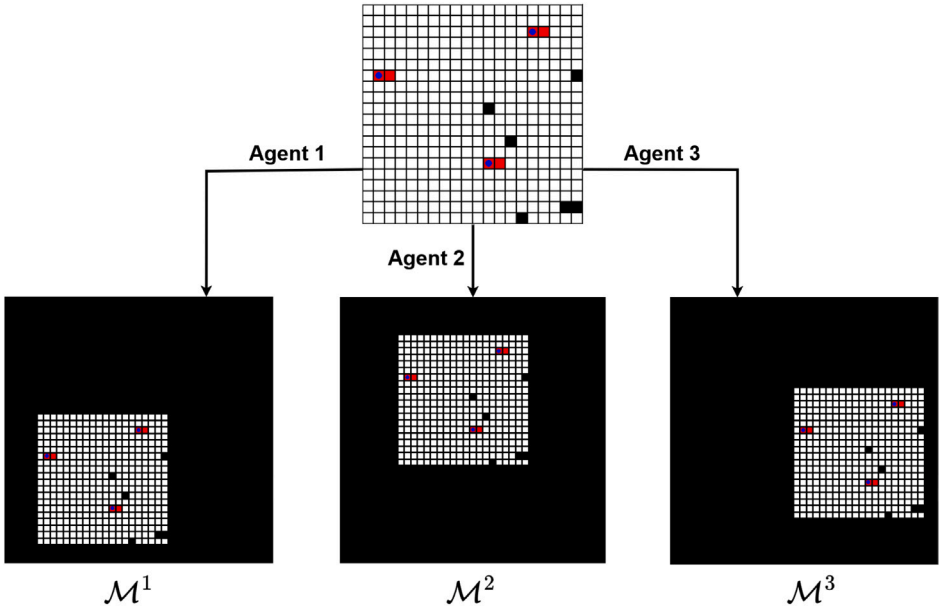


Fig. 3. The observation of the current state from the perspective of the different agents. One can always identify which agent the observation corresponds to by analyzing the center pixel.

However, it is important to note that the nature of the Stochastic Game makes the problem of state looping less prevalent. This is because the state evolves through the effect of the joint actions $a^1 \times a^2 \times \cdots \times a^j$ of all agents. For an infinite loop to occur, all agents' actions must simultaneously fail to alter the sequence of observations. However, from the perspective of any single agent, even a change in another agent's position is enough to modify its own observations. This dynamic interaction among agents reduces the likelihood of state loops forming.

Nonetheless, we will rely on our formulation to be compatible with the problem in a single-agent setting, so it is still necessary to mitigate this issue. In addition, all the following augmentations of the observation space also have their standalone purpose besides the state-looping problem.

Thus, our first augmentation is to include the battery level b^j in the observation o^j of the agent j. This variable serves as a way to give agents time awareness [54], which makes the observation change over time and helps with function approximation in finite Markov Decision Processes by diminishing the effects of episode truncation.

Furthermore, we also augment the observation by adding the last agent action $L_a^j \in \mathbb{B}^{|\mathbb{A}^j|}$ in the form of a vector that is a one-hot encoding of the Action Space $\mathbb{A}^j$. This gives the agent awareness of their previous action, working as a form of odometry. In a way, this extra information is a workaround to the Markov Property, and for path-planning problems, it can yield more rational paths, as their choice is also based on the path that was being taken up to that point and not just on the current position.

Finally, the last augmentation is Frame-Stacking [55,56]; this technique is part of the original work of the DQN but is mostly unutilized in the context of the CPP literature. The rationale behind stacking the last $N \in \mathbb{N}$ observation of the map $\mathcal{M}^j$ in a vector $(\mathcal{M}_t^j, \mathcal{M}_{t-1}, \dots, \mathcal{M}_{t-N+1})$, is similar to the usage of the previous actions, which is to give a short term memory to the agent. In the context of the multi-agent problem, it also allows the agent to visualize the policy of the others, reducing the issue of non-stationarity and giving better reasoning abilities to the agent. However, it is important to note that this formulation is not $N-$decodable [57], as there is no finite amount of short-term memory N that will make the whole state always recoverable.

Nonetheless, we extend the frame-stacking to the last action L_a^j and to the out-of-bounds information Ψ^j, and each agent only has access to their own observations. This results in the final observation space being described as:

$$\Omega^j = \underbrace{\mathbb{R}^{N\times 4\times (m_c'+2)\times (m_c'+2)}}_{\substack{\text{Map Representation}\\ \mathcal{M}^j}} \times \underbrace{\mathbb{N}_0^{N\times 4\times 2}}_{\substack{\text{Out of Bounds}\\ \text{Information}\\ \Psi^j}} \times \underbrace{\mathbb{B}^{N\times|\mathbb{A}|}}_{\substack{\text{Last Actions}\\ L_a^j}} \times \underbrace{\mathbb{N}}_{\substack{\text{Battery}\\ \text{Level}\\ b^j}} \tag{18}$$

Moreover, to visualize how the observations differ between agents consider Fig. 3.

4.2. Reward function

In this subsection, we will formulate various candidate reward functions. All reward functions share the same general structure, in such a way that $R^j : \mathbb{S} \times \mathbb{A}_\times \times \mathbb{S} \to \mathbb{R}$ maps the state transitions into a scalar reward value. Note that due to the problem being modeled as a POSG, each agent j has its own reward function R^j. Despite the agents being homogeneous in Action Space and sharing the same global goal, i.e, covering the map, we will explore three possible settings for their reward functions: Competitive, Cooperative, and Mixed [58].

Our formulation of the reward functions will be Event-Driven (ED) [59,60], where an event $e(t) : \mathbb{S} \times \mathbb{A}_\times \times \mathbb{S} \to \mathbb{B}$ is viewed as an occurrence that was triggered by a state transition $\langle s, a, s' \rangle$. They will be exclusively used for rewards, unlike the mentioned frameworks. Moreover, an event will always be attached to an agent, denoted by the superscript $j \in \mathbb{J}$.

To this end, we will define three types of events. These events are not mutually exclusive. First, we define an event $e_{ts}^j(t+1)$ that occurs at every state transition $\langle s, a, s' \rangle$ as follows:

$$e_{ts}^j(t+1) = 1 \ \forall t \in [0, T-1]. \tag{19}$$

This event penalizes the passage of time, modeling the minimization of time from the optimization framework. Furthermore, we define the collision $e_c^j(t)$ event as

$$e_c^j(t+1) = \mathbb{1}\left(\left\{ P_{t+1}^j \in \mathbb{O} \ \vee \ (\exists i \in \mathbb{J},\ i \neq j \ : \ P_{t+1}^j = P_{t+1}^i) \right\}\right) \tag{20}$$

which is triggered whenever there is a collision between agent j and the environment or any other agent i. Finally, we define the event e_n^j that is triggered whenever the agent j visits a new cell of interest:

$$e_n^j(t+1) = \mathbb{1}\left(\{ P_{t+1}^j \in \mathbb{C}_t \}\right). \tag{21}$$

The combination of these three events will serve as the cornerstone to construct all the prototype reward functions. Each event is accompanied by a gain $k_{ts}, k_c, k_n \in \mathbb{R}$, which modulates the importance given to each event. As in [53], we will set $k_n = -k_{ts} = 1$ in order to give the value function size invariance. The penalization of a crash is given as $k_c = -2$.

4.2.1. Prototype reward functions

We start by proposing an intuitive and simple prototype reward function as the starting point for all reward functions used throughout the document. Thus, consider:

$$R^j_{t+1} = k_n e^j_n + k_{ts} e^j_{ts} + k_c e^j_c + \frac{K}{J-1} \sum_{i=1, i\neq j}^{J} k_n e^i_n \tag{22}$$

In this function, the scalar gain $K \in [0, 1]$ is a tunable hyperparameter for the importance of other agents' performance on the agent j value function. This gain can be seen as the cooperativeness level of each agent, being that $K = 0$ is equivalent to a fully competitive reward function. On the other hand, higher values of K inherently reward collaboration. However, this can run into the issue of credit assignment, increasing the effects of the non-stationary problem [22] caused by giving more importance to a term of the reward that is directly dictated by other agents' policies.

Note that despite all agents' rewards sharing the same structure given by Eq. (22), $R^i \neq R^j, \forall i \neq j$. Moreover, for all $K \neq 0$, size invariance of the value function is lost as the cumulative reward can be larger than 0, due to the shared term. Nonetheless, focusing on the pure competitive setting where size invariance is kept, this stochastic game is a general sum with an expected value of zero, such that for environments with no obstacles, there exists a Nash Equilibrium (NE) solution where all agents visit the exact same value of cells of interest. In fact, this can almost be seen as a game that is globally zero-sum [61], as there is a finite amount of resources (cells of interest) that are to be split among the players of the game, with no changes in the amount of resources during the game that might change the outcome. Furthermore, the fact that an agent starts an episode with a cumulative reward of zero, and the optimal solution also has $v^*(s) = 0$, results in the fact that an agent cannot gain cumulative reward by harming the performance of others.

4.2.2. Size invariant value function

The next step is to modify the prototype reward function in order to keep the Size Invariant property. To deal with this issue, consider the following reward function:

$$R^j = (1-K) k_n e^j_n + k_{ts} e^j_{ts} + k_c e^j_c + \frac{K}{J-1} \sum_{i=1, i\neq j}^{J} k_n e^i_n. \tag{23}$$

In this case, the value function will still be size invariant for all $K \in [0, 1]$. This can be proved by the fact that in the case of an optimal policy where all agents visit a cell of interest, such that $e^j_n = 1,\ \forall j \in \mathbb{J}$, the Reward function reduces to:

$$R^j = (1-K) k_n e^j_n + \frac{K}{J-1} \sum_{i=1, i\neq j}^{J} k_n e^i_n + k_{ts} = (1-K) k_n + K k_n + k_{ts} = k_n + k_{ts} \tag{24}$$

resulting that for all $k_n = -k_{ts}$ the optimal value function is size invariant, yielding $v^*(s) \approx 0$. Note that this assumes that for all time steps, each agent always visits a cell of interest, which might be unrealistic for the multi-agent case. Nonetheless, this form of normalization is still valuable for handling domain shift, as regardless of the map size and number of agents, the optimal value will always be near zero.

4.2.3. Extension to Dec-POMDPs and cooperative settings

The prototype function that we presented can be used to model Stochastic Games with different objectives. If $K = 0$, the agents play a competitive game; for almost all values of K, the reward function models a mixed setting game, and for the specific case where $K = \frac{1}{J}$, we obtain a cooperative game.

Remark 1. In our framework, due to the individual penalization of collisions e^j_c, it is not possible to have agents that strictly follow the definition of cooperative [58]. Nonetheless, as this term will never be used in optimal policies, we will consider the setting to be cooperative if all other terms of the reward function yield the same reward value for all agents.

However, some MARL methods do not admit the flexibility of the POSG framework and require the problem to be modeled as a Dec-POMDP. In this work, we also consider that possibility, and adapt the reward structure to this paradigm whenever necessary. To this end, we apply a transform to the joint reward signal $\mathbf{R}$:

$$\mathbf{R} = \frac{1}{J} \sum_{j=1}^{J} R^j, \tag{25}$$

such that the team reward signal is the average of all individual reward signals, and agents receive the same reward. Note that by applying this transformation, the formulation loses the degree of freedom to allow for tuning the level of cooperativeness. In the case of the size invariant reward function (23), the team reward signal is the same independently of the value of K, keeping the size invariance property. On the other hand, for the non-size invariant case (22), by applying the transform, in the optimal case, one obtains:

$$\mathbf{R}^* = \frac{1}{J} \sum_{j=1}^{J} R^j = (1+K) k_n + k_{ts}. \tag{26}$$

Thus, for any value of $K \neq 0$, the value function loses the size invariant property for all $k_n = -k_{ts}$. In fact, for the Dec-POMDP, the hyperparameter K does not change the cooperativeness level. Rather, it modifies the relative importance assigned to visiting cells of interest through an additive adjustment.

4.3. Safety and robustness filtering

One of the most significant issues with the usage of Reinforcement Learning methods in real-world applications is the lack of safety and robustness guarantees. This problem is exacerbated in our case due to the focus on generalization, as it is harder to guarantee that the trained policies will always be adequate for every possible scenario. As our joint-observation space is prohibitively large, we will use the concept of safety-filtering [24] through action masking [62]. To this end, we will consider that our algorithm is safe as long as it is collision-free and robust, if the algorithm can always complete the task. i.e., state loops do not occur. Note that due to this fact, unlike the literature, we are not interested solely in a safety filter, as we also want to avoid undesirable scenarios (but not necessarily unsafe). Hence, we propose two filters, one for safety and one for robustness/completeness, in a cascade with two stages. The filter architecture can be visualized in Fig. 4, wherein the first stage, the robustness filter, guarantees that our policy will steer the agents to a set of desirable states, and then a safety filter will check for the safety of the overall policy for certifiable safe decision-making.

These filter components exist besides the Reinforcement Learning agents, working on a supervisory layer, meaning that a filter can control the agent, but not vice versa. In this manner, instead of considering the typical agent observation $o^j_t \in \Omega^j$, a filter has access to the information-states $\boldsymbol{\eta} \in \mathbb{H}_\times$, such that $\mathbb{H}_\times = \mathbb{H}^1 \dots \times \mathbb{H}^{J^*}$. In this work, we will use the information state $\eta^j_t \in \mathbb{H}^j$ as an observation with enough frames N' such that all filters are realizable. Furthermore, we want the

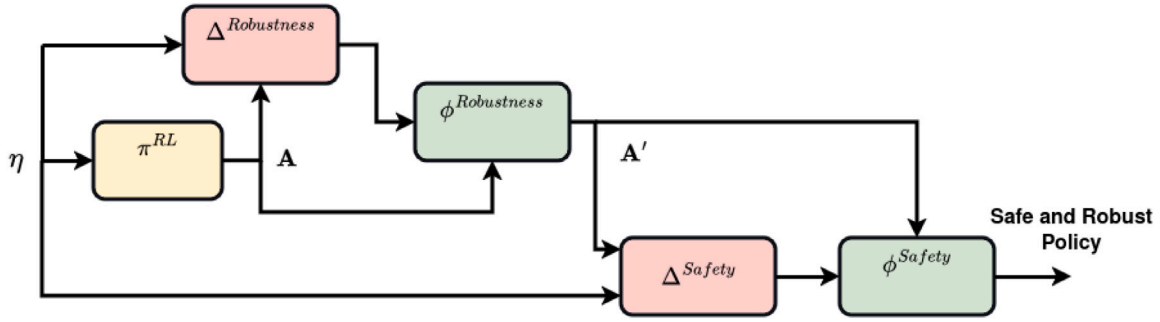


Fig. 4. Safety and Robustness Filter Architecture.

computation of the filter to be decentralized; as such, each agent must contain the filter structure and all the necessary information.

$$\mathbb{H}^j = \underbrace{\mathbb{R}^{N'\times 4\times(m'_c+2)\times(m'_c+2)}}_{\text{Map Representation } \mathcal{M}^j} \times \underbrace{\mathbb{N}_0^{N'\times 4\times 2}}_{\substack{\text{Out of Bounds} \\ \text{Information } \Psi^j}} \times \underbrace{\mathbb{B}^{N'\times|\mathbb{A}|}}_{\text{Last Actions } L_a^j} \times \underbrace{\mathbb{N}}_{\substack{\text{Battery} \\ \text{Level } b^j}} \tag{27}$$

Remark 2. This information state is a deviation from the Safety Filter literature and is used solely to keep the consistency of our Deep Reinforcement Learning Framework. Moreover, for the safety filter, the current observation would be enough, as there is enough information encoded for every situation. However, the more expansive information state enables a more powerful robustness filter.

In MARL applications and Decentralized Stochastic Games, an inherent conflict exists in how the agents communicate and share information. While Decentralized Stochastic Games assume no explicit communication between agents, they can implicitly communicate through the state/observation. However, in real systems, this implicit communication means that there must exist communication between agents or at least between the agent and a coordinator. To deal with this conundrum and keep the problem as decentralized and feasible as possible, we consider that the filter does not need to take into account global information but only local information. Thus, from the perspective of agent j, it is part of a group of agents $\mathbb{J}^*$ such that the set is defined as

$$\mathbb{J}^* = \{i \mid \forall i \in \mathbb{J} \setminus \{j\}, \|P^j - P^i\|_1 < d\} \tag{28}$$

where d is the maximum Manhattan distance between the agent j and the members of the group of size J^*. This makes our framework akin to an ad-hoc UAV network.

4.3.1. Robustness filter

We will start by formulating the Robustness Filter, which is the first stage of the architecture. The goal of this filter is to steer the agents so that their information state η_t^j is in a subset of desirable information states $\mathbb{H}^{j,d}$. With this goal, the first step is to define our robustness monitor.

Let $f_e(\boldsymbol{\eta}, \mathbf{A}) : \mathbb{H}_\times \times \mathbb{A}_\times \to \mathbb{H}_\times$ be a weak simulator [63], that can simulate observation transitions without access to the reward or the deterministic state[2] and $\mathbb{H}_\times^d \subset \mathbb{H}_\times$ be the set of joint desirable information states, where $\mathbb{H}_\times^d \implies \mathbb{H}^{j,d}$, $\forall j \in \mathbb{J}^*$ meaning that all individual agent observations are also desirable. To have access to this simulator, the agent must know both the other agents' information state and actions. In the case of the states, we assume that agents in the same group $\mathbb{J}^*$ have the same information, as communication is feasible due to proximity. As for the actions, while it would be possible to share directly the next action, this might cause delays in the decision-making process, as the agent must make a decision, wait for the other agents' decisions, and then coordinate the changes in actions. Instead, we will take advantage of the fact that the agents are trained through parameter sharing, and thus, they can have the same policy $\pi_{RL}^1 = \cdots = \pi_{RL}^j$ even during deployment, due to agent invariance.

Lemma 1. *Given a set of agents $\mathbb{J}^*$ which share the same deterministic policy $\pi^1 = \pi^2 = \cdots, \pi^{J^*}$, an agent i can predict the next action a_{t+1}^j of the agent j.*

Proof. If agents $i, j \in \mathbb{J}^*$, such that agent i has access to the exact observation o^2, then $\pi^1(o_t^2) = \pi^2(o_t^2) = \pi^{J^*}(o_t^2)$ [64].

Due to this equality, we can construct the vector $\mathbf{A} = [a^1, \ldots, a^{J^*}]$ with an arbitrary policy $\pi^j, \forall j \in \mathbb{J}^*$.

Definition 2. Given the system described in Section 3 and its weak simulator $f_e(\boldsymbol{\eta}, A)$, a function $\boldsymbol{\Delta}^{robust}(\boldsymbol{\eta}, \mathbf{A}) : \mathbb{H}_\times \times \mathbb{A}_\times \to \mathbb{B}^{J^*}$ is said to be a **robustness monitor** if it satisfies the implication:

$$\boldsymbol{\Delta}^{robust}(\boldsymbol{\eta}, \mathbf{A}) = \vec{\mathbf{1}} \implies f_e(\boldsymbol{\eta}, \mathbf{A}) \in \mathbb{H}_\times^d, \tag{29}$$

where, given a $k_p, k_s \in \mathbb{N}$, the individual elements of the output of the robustness monitor can be computed as follows

$$\Delta^{r,j} = \mathbb{1}\left(\left\{ \sum_{\tau=0}^{k_s} e_n^j(t+1-\tau) = 0 \right. \right. \\ \left. \left. \vee \sum_{\tau=0}^{k_p} e_n^j(t+1-\tau) = 0 \wedge \|P_{t+1}^j - P_{t+1-k_p}^j\|_1 \le 1 \right\}\right) \tag{30}$$

Remark 3. Despite the next observation $o_{t+1}^j \sim P(\cdot|o_t^j, A_t)$ given by the weak simulator containing a stochastic component, the critical information for agent j to visit is a cell of interest is local, satisfying $\|P_t^j - X\|_1 \le 1 \ \forall X \in \boldsymbol{\xi}$. This local component is deterministic, as it is fully observable, which in turn ensures that the monitor's guarantee remains deterministic.

The rationale behind the monitor is that the undesirable states $\eta_t^j \notin \mathbb{H}^{j,d}$ are when the agent j has not visited a new cell of interest in k_s timesteps or when it has not moved from the same position in k_p steps. Note that in general $k_p \le k_s$, as not moving from the current position is a more severe robustness violation, so it should be rectified earlier. Note that these states are not inherently unsafe, they are just undesirable and thus are decoupled from the safety guarantees. In addition, the value of k_s gives the upper bound of N to make the filter realizable. It is also important to note that the desirable set $\mathbb{H}^{j,d}$ might also not be reachable in one timestep. This can mean, for example, that the closest cell of interest is far away, requiring multiple steps to reach. Consequently, $\mathbb{H}^{j,d}$ is not a controlled-invariant set. As such, there is no guarantee that whenever the agent state is inside this set, there exists a

[2] Note that the weak simulator does not have any information besides the observation, as such the evolution of the map is through a likelihood function $o_{t+1} \sim P_r(.|o_t, A_t)$

policy such that the agent never leaves it. Instead, we define a fallback policy $\boldsymbol{\pi}^{fallback}$ such that whenever the state is outside the desirable set, the policy steers towards it.

Definition 3. A fallback policy $\boldsymbol{\pi}^{fallback} : \mathbb{H}_\times \to \mathbb{A}_\times$, a robustness monitor $\boldsymbol{\Delta}^{robust}(\boldsymbol{\eta}, \mathbf{A}) : \mathbb{H}_\times \times \mathbb{A}_\times \to \mathbb{B}^{J^*}$, and an intervention scheme $\boldsymbol{\phi}^{robust} : \mathbb{H}_\times \times \mathbb{A}_\times \to \mathbb{A}_\times$ constitute a **robustness filter** if the following implication is satisfied:

$$\boldsymbol{\Delta}_t^{robust}(\boldsymbol{\eta}_t, \mathbf{A}_t) = \vec{\mathbf{0}} \implies \exists \tau > 0 \text{ such that } \boldsymbol{\Delta}^{robust}(\boldsymbol{\eta}_{t+\tau}, A'_{t+\tau}) = \vec{\mathbf{1}}, \quad \text{if} \quad \mathbf{A}'_{\mathbf{k}} = \boldsymbol{\phi}(\boldsymbol{\eta}_k, \mathbf{A}_k),\ \forall k \in [t, t+\tau]. \tag{31}$$

Remark 4. While not explicitly stated, a terminal state $\boldsymbol{\eta}_T$ is also part of the set of desirable states $\mathbb{H}_\times^d$.

Our definition of robustness filter is that given the joint fallback policy $\boldsymbol{\pi}^{fallback}$, there is a finite amount of timesteps τ such that the joint information state $\boldsymbol{\eta}$ converges to the set of desirable states $\mathbb{H}_\times^d$. This convergence is achieved by using the following intervention scheme for each agent:

$$\phi^{r,j} = \begin{cases} \pi^{fallback,j} & \text{if } \Delta^{r,j} = 0 \\ \pi^{RL,j} & \text{Otherwise,} \end{cases} \tag{32}$$

where the switch-type intervention scheme chooses between the fallback $\pi^{fallback}$ or the action given by the Reinforcement Learning Policy π^{RL} depending on whether there is an anomaly.

4.3.2. Safety filter

With the robustness and completeness guarantees assured, the next step of the action filtering pipeline is to guarantee safety. Unlike the previous filter, the safety requirements are more stringent, as they can never be violated. It is also important to note that we will use the same joint fallback policy to guarantee safety, so this policy is also inherently safe. However, there is a need to recalculate the policy if at least one agent following the Reinforcement Learning policy violates the safety constraint, i.e., the joint fallback policy as a whole will change.

Consider now the safe set $\mathbb{H}_\times^{safe} \subset \mathbb{H}_\times$, which, to comply with the safety requirements, must be a controlled-invariant set. In practice, this means that for all agents, the joint information state $\boldsymbol{\eta}_t$ must be part of the safe set $\mathbb{H}_\times^{safe}$ for all $t \in [0, T]$, and there exists at least one policy so that the agents never leave the safe set. To guarantee safety at all times, we will formulate a safety filter, which is a more restrictive version of the previous robustness filter.

Definition 4. Given the system described in Section 3 and its weak simulator $f_e(\boldsymbol{\eta}, \mathbf{A})$, a function $\boldsymbol{\Delta}^{safe}(\boldsymbol{\eta}, \mathbf{A}) : \mathbb{H}_\times \times \mathbb{A}_\times \to \mathbb{B}^{J^*}$ is said to be a **safety monitor** if satisfies the implication:

$$\Delta^{safe}(\boldsymbol{\eta}, A) = \vec{\mathbf{1}} \implies f_e(\boldsymbol{\eta}, A) \in \mathbb{H}_\times^d. \tag{33}$$

The individual elements of the output of the safety monitor can be computed as:

$$\Delta^{s,j} = \mathbb{1}\{P_{t+1}^j(a_t^j) \notin \boldsymbol{\Lambda} \vee P_{t+1}^j(a^j) \neq P_{t+1}^i(a^i), i \neq j, \forall i \in \mathbb{J}^*\}, \tag{34}$$

which is to say that the failure set includes states where there is a collision between the agent and the environment or inter-agent collision.

Definition 5. A fallback policy $\boldsymbol{\pi}^{fallback} : \mathbb{H}_\times \to \mathbb{A}_\times$, safety monitor $\boldsymbol{\Delta}^{safe}(\boldsymbol{\eta}, \mathbf{A}) : \mathbb{H}_\times \times \mathbb{A}_\times \to \mathbb{B}^{J^*}$, and an intervention scheme $\boldsymbol{\phi}^{safe} : \mathbb{H}_\times \times \mathbb{A}_\times \to \mathbb{A}_\times$ constitute a **safety filter** if the following implication is satisfied:

$$\boldsymbol{\Delta}^{safe}(\boldsymbol{\eta}, \boldsymbol{\pi}^{fallback}(\boldsymbol{\eta})) = \vec{\mathbf{1}} \implies \boldsymbol{\Delta}^{safe}(\boldsymbol{\eta}, \boldsymbol{\phi}^{safe}(\boldsymbol{\eta}, \mathbf{A})) = \vec{\mathbf{1}},\ \forall \mathbf{A} \in \mathbb{A}_\times. \tag{35}$$

We establish that the safety monitor returns a vector of ones whenever a candidate joint action $\mathbf{A}$ keeps the joint information state in the safe set. Then, if our joint fallback policy $\boldsymbol{\pi}^{fallback}$ satisfies the safety monitor for all possible $\boldsymbol{\eta}$ information states, an adequate intervention scheme guarantees that the safe set is controlled-invariant. Once again, the intervention scheme is designed as follows:

$$\phi^{s,j} = \begin{cases} \pi^{fallback,j} & \text{if } \Delta^{r,j} = 0 \vee \Delta^{s,j} = 0 \\ \pi^{RL,j} & \text{Otherwise.} \end{cases} \tag{36}$$

Algorithm 1 Policy Algorithm

```
Estimate the joint information state η
Calculate safety and robustness monitor outputs Δ
Initialize vectors: Next Positions P_{t+1}, Goal Positions G, and Actions A
for each agent j = 1 to J* do
    if Δ^j = 1 then                                   ▷ Agent j is safe to follow RL policy
        A^j ← π^{RL}(η^j)
        P^j_{t+1} = P^j_t + A^j
    else                                              ▷ Follow Fallback Policy
        C^j_sorted ← Cells of Interest C_t sorted by minimum Manhattan Distance ‖c − P^j_t‖_1, ∀c ∈ C_t
        for each cell c_n ∈ C_sorted^j do             ▷ Check closest cells
            if c_n ∉ G and c ∉ P_{t+1} then            ▷ Cell is free
                Φ^j ← Dijkstra(P^j_t, c_n)
                c_0 ← Φ[0]                            ▷ The first cell of the path.
                A^j ← arg_a(P^j_{t+1}(a) = c_0),  s.t Eq. (6).
                P^j_{t+1} ← c_0, G^j ← c_n
                break                                 ▷ Stop after valid move
            else                                      ▷ Handle conflicts
                i ← agent with c_n = G^i or c_n = P^i_{t+1}
                if Δ^i = 0 then                       ▷ Other agent does not have priority
                    Assign cell c to the agent closest to it
                    if tie exists then
                        continue                      ▷ Skip to next cell
                    end if
                else
                    continue                          ▷ Skip to next cell
                end if
            end if
        end for
    end if
end for
return A
```

4.3.3. Fallback policy

The last piece of the architecture is the fallback policy. We will consider a joint fallback policy $\boldsymbol{\pi}^{fallback} : \mathbb{H}_\times \to \mathbb{A}_\times$ which is computed locally by every agent. This is due to the fact that this policy needs cooperation between agents, meaning that the agent i needs to know the intentions of the agent j, and this interaction changes the output of the policy. To get around communication constraints, each agent computes this joint policy, and since the rationale is the same for all agents, they can predict each other without explicitly communicating actions. To this end, consider the proposed Algorithm 1 for the joint policy.

The rationale behind this policy is that whenever an agent must rely on it, it will select the path from its position to the closest cell of interest. Moreover, there is a priority in the attribution of the cell of interest; the first criterion is that agents that are using the RL policy, meaning that $\Delta^j = 1$, always get priority. The second criterion is that whenever $i, j \implies \Delta^i = \Delta^j = 0$, and both agents select the same cell c_n, the one who gets priority is whoever is closest to it in Manhattan Distance. Finally, if there is a tie in distance, the agent with the lower index gets priority.

4.3.4. Safety, robustness and completeness properties

With the proposed safety and robustness filter and the proposed fallback policy, we can prove that our method is safe, robust, and complete.

First, we define safety of the system as having no agent–environment and inter-agent collisions for all $t \in [0, T]$. We can prove that our system state is always inside the safe set $\mathbb{H}_{\times}^{safe}$, with the following Theorem:

Theorem 1 (*Universal Safety Filter Theorem [24]*). *Consider the system described in Section 3, its weak simulator $f_e(\boldsymbol{\eta}, \boldsymbol{A})$ and a safety filter as in Definition 5, comprising a joint fallback policy $\boldsymbol{\pi}^{fallback}$ defined in Algorithm 1, a safety monitor $\boldsymbol{\Delta}^{safe}$ as in Definition 4, and an adequate intervention scheme $\boldsymbol{\phi}^{safe}$ as in Eq. (36). If the system initial condition $\boldsymbol{\eta}_0 \in \mathbb{H}_{\times}$ is deemed safe by the safety monitor under the fallback policy, that is $\boldsymbol{\Delta}^{safe}(\boldsymbol{\eta}, \boldsymbol{\pi}^{fallback}(\boldsymbol{\eta}_0)) = 1$, then its evolution under the weak simulator dynamics $f_e(\boldsymbol{\eta}, \boldsymbol{\phi}^{safe}(\boldsymbol{\eta}, \boldsymbol{\pi}^{RL}))$ satisfies the safety condition $\boldsymbol{\eta}_t \in \mathbb{H}_{\times}^{safe}$, $\forall t \in [0, T]$ for all possible Reinforcement Learning joint policies $\boldsymbol{\pi}^{RL}$.*

Proof. For a general proof of the theorem, refer to [24]. In the specific case of the multi-agent filter, let the joint actions $\boldsymbol{A}$ be determined according to the conditions stated in Lemma 1. Then the fallback policy $\pi^{fallback}$, which ensures agent–environment collision avoidance using Dijkstra's algorithm, also guarantees inter-agent collision avoidance due to the constraints enforced in line 11 of Algorithm 1. Since the intervention scheme $\boldsymbol{\phi}^{safe}$ ensures that any action violating safety constraints is overridden by a collision-free alternative, the system state remains within the safe set $\mathbb{H}_{\times}^{safe}$ at all times.

With this setup, we obtain our safety certificate by guaranteeing that all monitor checks are valid. Note that the safety filter may never apply the fallback policy as it is supposed to be intrusive only when it is strictly necessary.

As for the completeness certificate, consider the following theorem:

Theorem 2 (*Completeness Certificate*). *Consider the system model presented in Section 3 and assume that the battery level $b^j, \forall j \in \mathbb{J}$ is a sufficiently large positive integer. Then given a robustness filter as per Definition 3, which follows the intervention scheme $\boldsymbol{\phi}^{robustness}(\boldsymbol{\eta}, \boldsymbol{\pi}^{RL}(\boldsymbol{\eta}))$ as described in Eq. (32), with the fallback policy $\pi^{fallback}$ given in Algorithm 1, it is guaranteed that the terminal condition $\mathbb{C}_T = \emptyset$ is reachable for every feasible initial condition $\boldsymbol{\eta}_0 \in \mathbb{H}_{\times}$, that is, the policy is* ***Complete****.*

Proof. For detailed proof of **Completeness**, refer to [53]. In the multi-agent scenario, the argument follows similar logic. The fallback policy $\pi^{fallback}$ is complete for a sufficiently large battery level b_t^j. This is due to all cells $c \in \mathbb{C}$ being globally reachable as per Definition 1. Thus, the problem can be solved within a finite amount of $|\mathbb{C}_0|$ decisions, where the decision is to compute the shortest path $|\boldsymbol{\Phi}(c)|$ to the closest cell $c \in \mathbb{C}_t$. Then, a sufficiently small k_p makes it so that the robustness monitor $\boldsymbol{\Delta}^{robustness}$ triggers frequently enough so that completeness is guaranteed even in the worst-case scenario.

Finally, we analyze the robustness properties of our method. We define robustness as the system's ability, given the current information state $\boldsymbol{\eta}_t \notin \mathbb{H}_{\times}^{d}$, to recover to the desirable set $\boldsymbol{\eta}_{t+\tau} \in \mathbb{H}_{\times}^{d}$ within a finite amount of timesteps $0 < \tau \leq T - t$. In practice, this ensures that if an agent exhibits non-robust behavior, such as being stuck in a state loop or failing to visit a cell of interest $c \in \mathbb{C}$ for more than k_s timesteps, our robustness filter intervenes by modifying the policy. This adjustment directs the agent(s) to desirable states, ensuring they continue performing the task as intended.

Theorem 3 (*Robustness Certificate*). *Consider the system model presented in Section 3 and assume that the battery level of agent j, denoted as b^j, is sufficiently large. Suppose that at time t, the agent's information state satisfies $\eta_t^j \notin \mathbb{H}_d^j$, meaning it is outside the desirable set. Given a robustness filter as per Definition 3, with the robustness monitor $\boldsymbol{\Delta}^{robustness}$ described in Definition 2, and following the intervention scheme $\boldsymbol{\phi}^{robustness}(\boldsymbol{\eta}, \boldsymbol{\pi}^{RL}(\boldsymbol{\eta}))$ from Eq. (32), which incorporates the fallback policy $\pi^{fallback}$ described in Algorithm 1, the robustness filter guarantees that the agent's information state $\eta_{t+\tau}^j$ will reach the desirable set $\mathbb{H}_d^j$ within a finite number of time steps τ, which satisfies $0 < \tau \leq T - t$.*

Proof. Let the robustness filter be active, $\Delta^{j,robustness} = 1$, ensuring that agent j follows the fallback policy $\pi^{j,fallback}$. The fallback policy directs the agent towards the nearest unvisited cell of interest $c \in \mathbb{C}_t$ along the shortest path $\boldsymbol{\Phi}(c)$. The number of steps required to reach c is $|\boldsymbol{\Phi}(c)|$, which exists and is finite due to all cells being globally reachable as per Definition 1. If another agent i has already visited c, then agent j selects the next closest unvisited cell in $\mathbb{C}_t$. This process continues until either an unvisited cell is reached, in which case the agent enters $\mathbb{H}_d^j$, or all cells in $\mathbb{C}_t$ have been visited, meaning that $\eta_{t+\tau}^j$ is necessarily in $\mathbb{H}_d^j$. The latter case is guaranteed since the method is complete as per Theorem 2. Since the number of cells in $\mathbb{C}$ is finite, the task always terminates in a finite time τ, satisfying $0 < \tau \leq T - t$.

5. Multi-agent reinforcement learning

In this section, we introduce our Multi-Agent Reinforcement Learning framework used to solve the proposed POSG/Dec-POMDP. First, we describe the theoretical framework of Q-learning and its extension to multi-agent scenarios through centralized training, decentralized execution, and parameter sharing. Finally, the neural network architecture, hyperparameters, and learning algorithm are described.

5.1. Q-learning

Reinforcement Learning (RL) methods are commonly used to solve Markov Decision Processes (MDP) [32], which are the single-agent formulation of Stochastic Games/Markov Games [58]. From the RL literature, one of the cornerstone methods is Q-Learning [31], which is a model-free, value-based method.

Consider a problem that is described between the sequential interaction between an agent (or multiple) and the environment, where at each time-step t the agent observes the state $s_t \in \mathbb{S}$, takes an action $a_t \in \mathbb{A}$, to which the environment responds with a scalar reward $R_{t+1}(s_t, a_t, s_{t+1})$ and its state evolves to s_{t+1}, and the cycle continues until a terminal state s_T.

The goal of RL methods is to learn a policy $\pi(a|s) : \mathbb{S} \times \mathbb{A} \to \mathbb{R}$ such that the agent maximizes their reward, i.e., achieving their goal in an optimal manner. Thus, we can model a value function $v_\pi(s)$, which is the inherent value, as in expected discounted cumulative reward, of each state s if the agent is following the policy π:

$$v_\pi(s) = \mathbb{E}_\pi\Big[\sum_{k=0}^{T} \gamma^k R_{t+k+1} | S_t = s\Big]. \tag{37}$$

Moreover, one can also formulate a state–action value function $Q_\pi(s, a)$, such that it represents the value of being state s and taking action a and then follow policy π:

$$Q_\pi(s, a) = \mathbb{E}_\pi\Big[\sum_{k=0}^{T} \gamma^k R_{t+k+1} | S_t = s, A_t = a\Big]. \tag{38}$$

Considering the case where the policy is deterministic $\pi(s) : \mathbb{S} \to \mathbb{A}$, due to the properties of the Bellman Optimality equation, this function can be expressed in a recursive manner such that:

$$Q_\pi(s, a) = R_{t+1} + \gamma Q_\pi(s_{t+1}, a_{t+1}) \tag{39}$$

With this result, one can arrive at the conclusion that the optimal policy is the one where the agent greedily chooses the action that maximizes the state–action value function:

$$\pi^* = \underset{a}{\operatorname{argmax}}\, Q_\pi^*(s, a). \tag{40}$$

Thus, the decision-making problem is reduced to estimating the state–action value function $\hat{Q}(s,a)$. To this end, in this work, we use neural networks as a function approximation, relying on the concepts introduced by the Deep-Q Network (DQN) [56]. In this method, the Deep Neural Networks estimate the value function $Q_\theta(s,a)$, where θ are the learnable parameters of the network. The vanilla version of the DQN algorithm uses two networks, the online with parameters θ, and the target θ^-, which is an offline copy that is updated by copying the online network parameters at every Γ steps. The online network weights are updated by sampling batches of transitions $e = (s_t, a_t, R_{t+1}, s_{t+1})$ from a replay buffer D and performing stochastic gradient descent to minimize the loss, defined as the squared TD error δ:

$$L(\theta) = \mathbb{E}_{e\sim D}\left[\left(R_{t+1} + \gamma \max_{a'} Q_{\theta^-}(s_{t+1}, a') - Q_\theta(s_t, a_t)\right)^2\right]. \tag{41}$$

5.2. Multi-agent Q-learning

The introduced Q-Learning algorithm has been explored in Multi-Agent Settings in the literature [65], and it has even been shown that agents with Q-Learning rationale can learn to coordinate action without being aware of each other [66]. Applications of multi-agent Tabular Q-Learning to coverage path planning have also been explored in a distributed manner [34]. A common paradigm of Q-Learning in multi-agent systems is Joint-Action Learning (JAL), also known as Centralized Q-Learning (CQL) [67], where the goal is to learn a centralized evaluation of the Q-function Q_c, which leads to an optimal joint action $a^*_\times \in \mathbb{A}_\times$. However, learning a joint policy is a difficult task and typically does not scale, as the joint action space grows exponentially. In practice, it makes it difficult to explore it and commonly leads to the lazy agent problem [64]. The other most common approach is Independent Learning, where agents learn their individual value-function Q^j, without considering the presence of others, including observations and policies. In this case, the agent j learns its individual policy π^j despite both the reward and state transitions depending on the joint actions [68]. This makes the learning problem difficult due to the non-stationarity of the environment caused by the agent's policy changing over time, making coordination and cooperation especially challenging.

In this work, our approach lies between CQL and IQL. The cornerstone of the framework is Centralized Training Decentralized Execution (CTDE) [21] with Parameter Sharing [69]. In this paradigm, during learning, all agents share a Q-Network $Q^1_\theta = Q^2_\theta = \cdots = Q^J_\theta$, and the implemented experience replay contains the individual observations of all agents. However, unlike CQL, there is no joint policy $\pi_\times = \text{argmax}_{\mathbf{a}}\, Q_\theta(\mathbf{o}, \mathbf{a})$, instead each agent follows its own policy $\pi^j = \text{argmax}_{a^j}\, Q^j_\theta(o^j, a^j)$, where the set of parameters θ is the same for agents, differing from IQL.

This approach can be classified as a parameter-shared and experience-shared Q-Learning [67]. This is possible since the agents are homogeneous and share action spaces, reward structures, and goals. The combination of shared experiences and parameters greatly helps alleviate the issues of non-stationarity associated with the MARL paradigm [22]. Moreover, the fact that the observation is always centered on the corresponding agent, which means that we do not require additional identification or agent markers [19] to identify and assign credit and actions. In practice, this enables distributed execution and scalability, as a centered observation of the agent o^j_t is sufficient to obtain the corresponding action a^j_t.

5.2.1. Parameter shared rainbow DQN

From this formulation of the learning problem, we implement and deploy two different methods that rely mostly on the same machinery. The first is adapting the Rainbow DQN [23], a popular method in the single-agent RL literature, to the MARL paradigm. This is done through Parameter Sharing and Experience Sharing. The main difference to the single-agent version is that the experience replay D contains the experiences of all agents, and the loss function is written as:

$$L(\theta) = \mathbb{E}_{e\sim D}\left[\left(R^j_{t+1} + \gamma \max_{a'} Q_{\theta^-}(o^j_{t+1}, a') - Q_\theta(o^j_t, a^j_t)\right)^2\right], \tag{42}$$

where j can be any agent in $\mathbb{J}$, and all agents use Q_θ for their policy.

5.2.2. Value decomposition networks

While the previous method is more of an adaptation of a single-agent method to a multi-agent setting, there are methods that were built to explicitly address the challenges of the latter. In this work, we also adapt and test our framework with Value Decomposition Networks (VDN). As a value-based method, it has many similarities to the previous method. However, the key difference is that the VDN estimates a global Q-Function $Q_{tot}(\mathbf{o}, \mathbf{a})$, relying on a shared reward signal $\mathbf{R}$, restricting it to Dec-POMDPs. The global Q-Function is estimated as:

$$Q_{tot}(\mathbf{o}, \mathbf{a}) = \sum_{j=0}^{J} Q_\theta(o^j, a^j) \tag{43}$$

And the loss function is instead rewritten as:

$$L(\theta) = \mathbb{E}_{e\sim D}\left[\left(\mathbf{R}_{t+1} + \gamma \max_{a'} Q_{tot}(\mathbf{o}_{t+1}, \mathbf{a}') - Q_{tot}(\mathbf{o}_t, \mathbf{a}_t)\right)^2\right]. \tag{44}$$

Note that in this formulation, applying the max operator to Q_{tot} is the same as applying the operator to each agent value function Q^j, keeping the policy decentralized. Moreover, as in the previous case, all agents share the same Q-Network, enabling CTDE.

The method learns a linear value decomposition from the global reward signal by back-propagating the global Q_{tot} gradient through the deep neural network that represents the individual value functions, enabling learning in a more centralized fashion, which helps with coordination [64]. Another method that follows the same key ideas is QMIX [70]. Instead of learning a linear combination of value functions, QMIX introduces a mixing network that receives as input the individual value functions and outputs Q_{tot}. This means that QMIX can approximate a richer class of functions due to the nonlinearities of the mixing network, not being limited to additive compositions.

5.3. Neural network architecture and learning algorithm

In order to approximate the Q-Function, we go beyond the vanilla DQN implementation and use a modified version of the Rainbow DQN [23] as our function approximator. From the original version, we removed the n-step learning and the distributional RL [71]. In the case of n-step learning, the choice is due to the increased variance that it causes, which is exacerbated in the multi-agent case due to the non-stationary properties of the agents' policies. As for the ablation of the distributional RL, it was due to the difficulty in tuning non-Atari examples and not being sample efficient [72]. The rest of the components: Double DQN, Dueling Networks, Noisy Linear Layers, and Prioritized Experience Replay, are implemented as in the canonical Rainbow DQN. In the case of the VDN method, we replace the Noisy Layers with a ϵ-greedy exploration strategy.

Moreover, the parameter update scheme is improved by using the soft-update from DDPG [73]. This technique is implemented by copying only a part of the target network parameters. The process can be parameterized by the hyperparameter $\tau \in [0,1]$ resulting in the expression:

$$\theta^- = (1-\tau)\theta^- + \tau\theta, \tag{45}$$

where τ is the importance given to the more recent parameters, where $\tau = 1$ would be the equivalent to the vanilla hard update.

The implemented neural network architecture can be visualized in Fig. 5. It is a Convolutional Neural Network (CNN), consisting of two main stages. The first is the feature extraction, consisting of three 2D convolutional layers that receive as input a reshaped Map Representation $\mathcal{M}^j \in \mathbb{R}^{12\times41\times41}$. In the case of using the neural network to

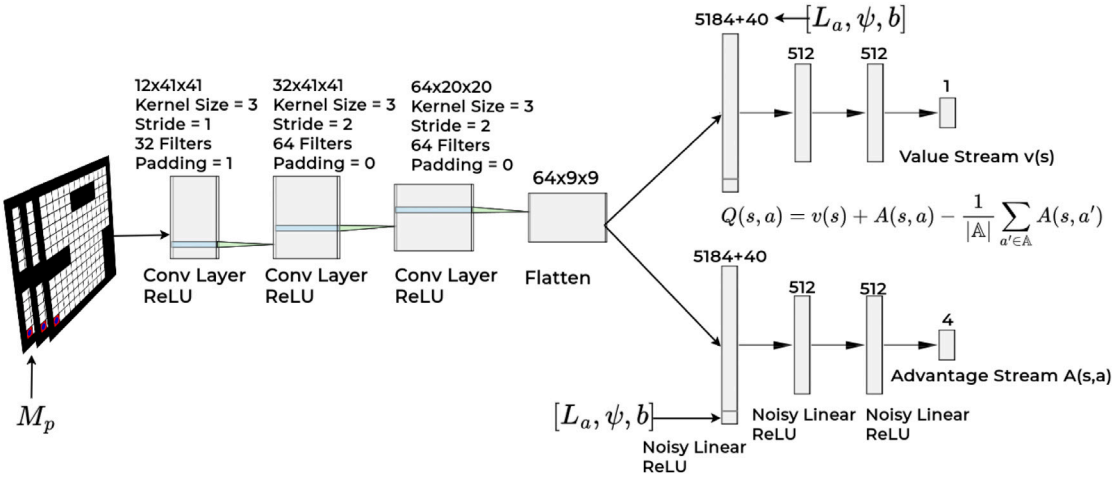


Fig. 5. The designed neural network architecture.

sample multiple policies, the input is the batch of map representations $\mathcal{M} \in \mathbb{R}^{J\times 12\times 41\times 41}$, where J is the number of policies/agents. The second stage of the architecture is built with two streams of noisy linear layers[3] [74], constituting the Dueling Architecture, which outputs the value-function $v_\theta(s)$ and the Advantage Function $A_\theta(s,a)$. The state–action value function estimation is obtained with the expression proposed by [58]:

$$Q_\theta(s,a) = v_\theta(s) + A_\theta(s,a) - \frac{1}{|\mathbb{A}|}\sum_{a' \in \mathbb{A}} A_\theta(s,a') \tag{46}$$

The architecture is very similar to the single-agent version [53], since the task is mostly the same, and it should be compatible with single-agent scenarios. Also, it opens the possibility of using the single-agent model as a baseline for the multi-agent task with transfer learning.

5.3.1. Training pipeline

Along with the architecture, we also developed a custom training pipeline that generalizes the one presented in [53] to multi-agent. The hyperparameters of the training algorithm are listed in Table 2. In this work, the main change is that the soft-update factor τ is smaller when compared to the single-agent version, which is necessary to stabilize training.

Another adaptation is to begin training with a pre-trained model that is already an expert in single-agent CPP, by using Transfer Learning. The core motif is to reduce the effects of non-stationarity and speed up training. If the agents are already capable of doing the task in single-agent settings, then cooperativeness is also easier to learn.

In the same spirit, we also introduce more design choices when it comes to the number of agents J. At the beginning of each episode e, we can randomly sample the number of agents, such that:

$$J_e = \begin{cases} 1, & \text{w.p} \quad \chi \\ \text{Uniform}([2, J_{max}]), & \text{w.p} \quad 1-\chi, \end{cases} \tag{47}$$

where J_{max} is a hyperparameter that specifies the range of the number of agents in the training set, and χ is a hyperparameter controlling the likelihood of selecting $J_e = 1$. In subsequent sections, we analyze the impact of these hyperparameters on model performance. We theorize that proficiency in the task with a different number of agents helps with generalization and reducing overfitting. Note that in the case of VDN/QMIX this change is not possible due to the structural assumption that the number of agents does not change during training.

[3] Regular Linear Fully Connected Layers for VDN.

Table 2
Neural network hyperparameters.

Parameter	Value
Minimum Steps Before Learning T_{start}	5000
Soft Update Factor τ	0.01
Discount Factor γ	0.999
Batch size B	32
Optimizer	ADAM
Loss Function	Smooth L1 Loss
ADAM Learning Rate η	0.00002
ADAM ϵ	0.000156
Maximum Gradient L2 Norm	2.5
Priority exponent α	0.5
Priority correction β	$0.4 \rightarrow 1$
Noisy Net Initial std deviation σ	0.5
Memory size $\lvert D \rvert$	3000000
Learning Period Γ	4
Number of Stacked Frames N	3
ϵ decay[a]	$0.8 \rightarrow 0.05$ In half of a task duration

[a] VDN Only.

As the Q-Learning is an off-policy algorithm, we can use the fallback policy $\pi^{fallback}$ as a form of imitation learning, introducing inductive biases to the learning process. To this end, there is no distinction between actions selected by either policy, as the transitions are stored in the experience replay, and the model can learn from this policy that has both explicit communication and expert information. We also employ Objective Optimization by using Curriculum Learning. The first step is to train an agent specialist in single-agent CPP and use Transfer Learning to train the model in the multi-agent scenario. In this work, we start with a pre-trained model obtained by following the methodology from [53]. Then, we employ a sequence of randomly generated training environments based on the hyperparameters in Table 3. A training set begins with the smaller and easier maps and builds up to the larger ones as the training progresses. The training algorithm pseudo-code can be visualized in Algorithm 2.

Note that during training, the safety filter is not used. This design choice is to enable the agents to be aware of the obstacles and the other agents. If there are no collisions during the training process, the RL policy itself might completely ignore obstacles, which in turn can make the trajectories inefficient and completely reliant on the safety

Table 3
Environment configurations for curriculum training.

Parameter	env1	env2	env3	env4	env5	env6
Steps (M)	2	2	2	2	4	4
Max Number Agents	2	2	2	2	2	2
Single Agent Probability χ[a]	50%	50%	50%	50%	50%	50%
Max Size	5	10	15	20	25	30
Min Size	4	5	10	10	15	4
Number Obstacles	5%	5%	5%	5%	5%	5%

[a] χ=100% for VDN/QMIX.

Algorithm 2 Curriculum Learning multi-agent CPP Training

Require: Environment configurations E, leaning period Γ, memory size M, PER exponents β, Learning Start Step T_{start}
Initialize replay memory D, PER exponent β and its increment Δ_β, online network θ and target network θ^-
for environment e in E **do**
  Initialize environment e, training budget T_{steps}
  for $t = 1$ to T_{steps} **do**
    Use online network θ to choose the batch of actions $\boldsymbol{A}_t$
    Filter the batch of actions $\boldsymbol{A}_t$ with the robustness filter and get $\boldsymbol{A}'_t$
    Take Action $\boldsymbol{A}'_t$, observe $\boldsymbol{O}_{t+1}, R_{t+1}$ and store all transitions
    if $t > T_{\text{start}}$ and $t\%\Gamma == 0$ **then**
      Reset Noisy Linear Weights / Update ϵ value
      Sample k transitions from the replay buffer
      Compute loss
      Perform Gradient Descent to update online network weights θ
      Soft Update the target network weights θ^-
    end if
    $\beta \leftarrow \beta + \Delta_\beta$
    if S_t is Terminal **then**
      Reset the environment
      Randomly sample the number of agents with probability χ
    end if
  end for
end for

filter taking action. On the other hand, the robustness filter can help the agents not get stuck in uninteresting parts of the State Space, greatly improving the stability and convergence during training.

6. Evaluation

In this section, we will evaluate the effectiveness of our framework in multiple scenarios. It begins by describing the conducted experiments and the performance metrics. Then, we perform ablation tests and study the effect of hyperparameters on the methods' performance. Finally, we test the framework in zero-shot generalization scenarios for different numbers of agents, map sizes, and the number of obstacles.

6.1. Evaluation scenarios and performance metrics

The focus of our paper is on the design of zero-shot generalizable and robust algorithms for Multi-Agent CPP. As such, our evaluation scenarios will mainly focus on the generalization capabilities of the method, as its effectiveness will be evaluated through Monte Carlo Simulations.

The models are first evaluated in randomly generated scenarios with similar distributions to the ones used during training. As a base, we use three ZSG datasets where each dataset corresponds to maps where the number of obstacles $\{0\%, 5\%, 10\%\}$ of the total cell count. Each dataset consists of 40 subsets of maps organized by size, ranging from $m \in [10, 50]$. Each subset contains 50 maps.

We build on top of this generalization dataset by augmenting it with more agents. Specifically, each map contains up to J_{max} versions, differing only in the position of the newly added agent between the Jth and $(J+1)$-th versions. This process can be visualized in Fig. 6.

To evaluate the performance, we will consider two metrics. The first is the Time Save Factor:

$$\lambda_J = \frac{T}{|\mathbb{C}_0|}, \tag{48}$$

where T is the terminal timestep, $|\mathbb{C}_0|$ is the number of points of interest at the initial state s_0, and J is the number of agents for that episode. As this metric optimal value varies for different numbers of agents J, when referring to a specific solution, we use the notation λ_J; moreover, the average value is referred to as $\hat{\lambda}$. Considering that in a single-agent scenario, the best case is when $T = |\mathbb{C}_0|$, implying that at every timestep, the agent visited a new cell of interest. So, this metric introduces an easy reference point for good solutions where the optimal value of $\lambda^\star_J = 1/J$. The other metric will be the Overlap of the solution:

$$O_J = \frac{T - \frac{|\mathbb{C}_0|}{J}}{\frac{|\mathbb{C}_0|}{J}}. \tag{49}$$

While both metrics are correlated, unlike the time save factor λ, the Overlap O_J has a lower bound of zero for all numbers of agents J, always representing the percentage of actions that did not lead to visiting a new cell of interest (path overlap).

Thus, the time-save factor λ_J allows one to evaluate the solution's efficacy in relation to the theoretical optimal for the number of agents, whereas the Overlap O_J evaluates how efficient the overall solution is. This allows us to draw conclusions about the effects of increasing the number of agents on both the efficiency and efficacy of the average solution.

6.2. Ablation tests for parameter shared rainbow DQN

At first instance, we start by analyzing the design choices exclusive to the Rainbow DQN method. In particular, we analyze the effects of χ on the method's performance during deployment, and the impact of transfer learning and curriculum learning on the performance. Regarding χ, this parameter cannot be used in the VDN as it is assumed that the number of agents per episode is always fixed. This also denies the possibility of starting training with a model that is already an expert in single-agent CPP, which also made it impossible to train the VDN without a curriculum, as the Q-function never converged.

To train the ablated models, we consider a training set based on Table 3 without any further modifications, unless the case of the ablation of curriculum learning, where the model is trained solely on the env6 settings with 16 million steps. The reward function is set to be Size Invariant (SI) with K=0.25, and we compare the ablations with the baseline trained with the same settings ($\chi = 0.5$) and a baseline VDN with the same curriculum and reward function. The average $\hat{\lambda}_2$ for the Monte-Carlo runs can be visualized in Fig. 7. Interestingly, the factor with the most impact is not randomly sampling the number of

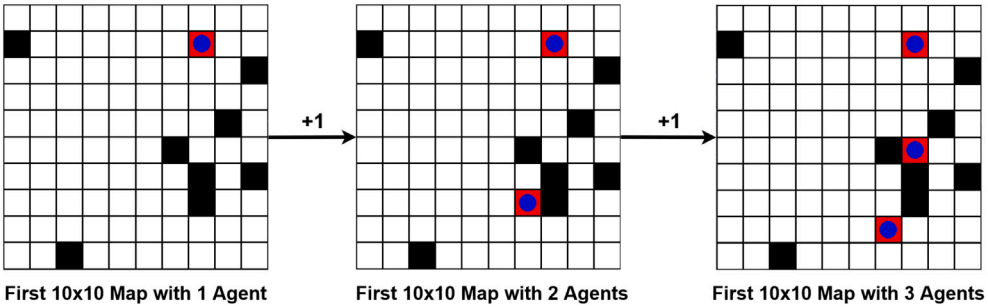


Fig. 6. The process of adapting maps to different numbers of agents.

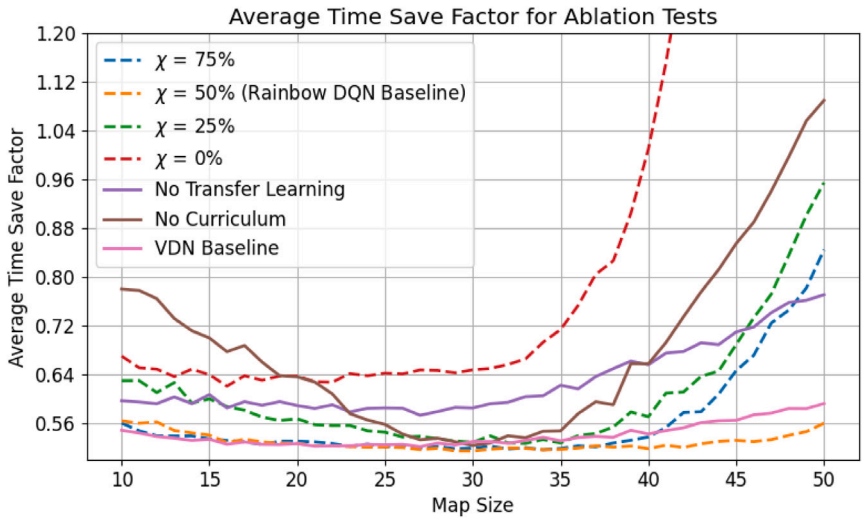


Fig. 7. Average λ for the Ablation Tests that were conducted.

agents, meaning that $\chi = 0$. Not only is the performance generally worse than the baseline, but it also fails to generalize to environments not seen during training $m > 30$. Thus, it is clear that being trained in single-agent scenarios is beneficial to the multi-agent setting.

As for the ablation of the Curriculum Training, interestingly, the performance is very similar for $m \in [25, 35]$, but it fails to generalize to the other map sizes despite training for the same number of time steps. Thus, it becomes evident that curriculum learning is essential to obtain a model with zero-shot generalization capabilities, as it only obtains good performance in the neighborhood of maps that were seen during training.

This is further shown in the Transfer Learning ablation scenario, where instead of starting with a model that is proficient in single-agent CPP, the method learns from scratch. In this scenario, there is a 10 to 20 percentage point increase in the average $\hat{\lambda}_2$ values compared to the Rainbow DQN baseline. This difference can be attributed to an increase in sample efficiency by using the pre-trained model, and it is possible that by increasing the training duration, the transfer learning ablation would converge to the baseline solution.

Changing the focus to the χ hyperparameter, the results from the evaluation can be seen in Fig. 7. The main takeaway from the results is that training with just a single agent is crucial for performance in the multi-agent setting. By ablating this parameter ($\chi = 0$), not only is $\hat{\lambda}_2$ consistently larger than for other values of χ, but also, the model completely fails to generalize to maps larger than 30×30, i.e., sizes that were never seen during training. As for the other values of χ, when $\chi = 0.25$, the performance is similar to $\chi = 0.5$ and $\chi = 0.75$ in maps of the range of $m \in [25, 35]$, but struggles to generalize to other map sizes. On the other hand, $\chi = 0.75$ is tied for the best performance up to map sizes of 40 but cannot generalize to maps much larger than the ones seen during training. Finally, the best results are obtained with $\chi = 0.5$, as it achieves the best balance between both ends of the spectrum. It can generalize to maps of a smaller size, like $\chi = 0.75$, while still being able to generalize to larger maps. We theorize that a larger value of χ can help with the cooperation/coordination at the local level, which is more valuable at smaller maps (the agents are closer), whereas a smaller value of χ enables the agent to be effective in single-agent CPP which in turn can improve the results where agents are more spread out, and cooperation is more at a macro level. In the remainder of the paper, all Rainbow DQN methods use $\chi = 0.5$.

Interestingly, despite not relying on transfer learning and the χ hyperparameter, the VDN baseline does not perform much worse than the Rainbow DQN, and is better than all the other ablation tests. One promising research direction can be to adapt the VDN methods to handle these augmentations and see if there are noticeable performance improvements, as in the case of the Parameter Shared Rainbow DQN.

6.3. Rainbow DQN vs VDN vs QMIX

In this subsection, we start by analyzing the difference in performance between the Parameter Shared DQN and the SOTA MARL methods, VDN and QMIX. The methods are trained in the curriculum presented in Table 3, and the network architecture is the same for all cases. The VDN and QMIX implementation follows the torchRL [75] framework, and the QMIX mixing network has a hidden size of 64.

We start the comparison by analyzing the results on the i.i.d 5% obstacle dataset, with different numbers of agents. The results can be visualized in Fig. 8. Interestingly, QMIX performed the worst of all the methods, especially in the scenario with two agents. We theorize that due to how the reward function and the problem were modeled,

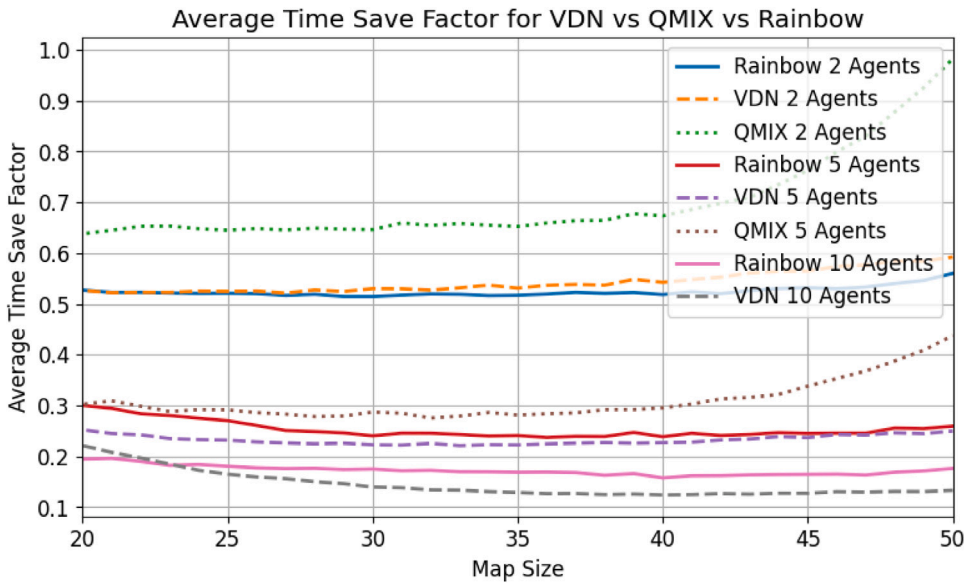


Fig. 8. Comparison between VDN, QMIX, and Rainbow DQN.

the value function can be decomposed linearly. Thus, having a more powerful non-linear representation may not be advantageous. It is also possible that the number of samples of the curriculum is not adequate for QMIX.

Nonetheless, the performances of VDN and the Rainbow are more similar. Indeed, the Rainbow performs better in scenarios with 2 agents but considerably worse when it comes to 5 and 10 agents. It is important to consider that in the scenario of 10 agents, decreasing 0.05 points in $\hat{\lambda}_{10}$, is more significant than in $\hat{\lambda}_2$. In the case of 10 agents, increasing the optimal time save factor $\lambda^*_{10} = 0.1$ to a solution with $\lambda_{10} = 0.15$ is equivalent to increasing the length of the path by 50%, whereas the same 0.05 increment with 2 agents is equivalent to increasing the path by just 10%.

This dichotomy of the Rainbow working better with fewer agents can be due to the method's training and formulation. For one, the Rainbow model trained in episodes with a singular agent, which are much more similar to the case when $J = 2$ than $J = 10$. Moreover, it is also important to consider that cooperation is more important as the number of agents increases. This, alongside the fact that the VDN learns a centralized value function, can justify this significant difference in performance.

6.3.1. Reward function and size invariant value function

In this paper, we introduced two reward function structures: the SI structure, which gives size invariance [53] to the value function, whereas the other is not size invariant, referred to as NSI. To study this novel reward function's effectiveness, we conduct two sets of tests: one with the Parameter Shared Rainbow and the other with VDN. Moreover, all the tests are conducted on the i.i.d 5% obstacles dataset.

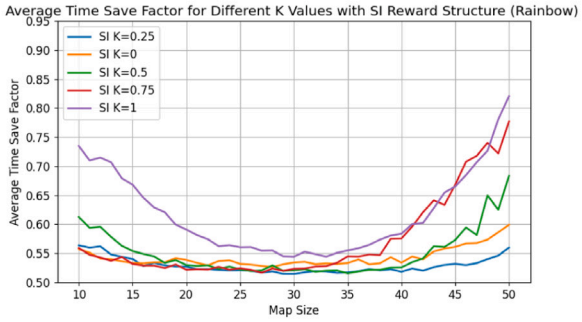
The results of the comparison between Size Invariant and Non-Size Invariant can be visualized in Fig. 9. Starting the analysis with the case of the Rainbow DQN with SI reward, in Fig. 9(a), it is noticeable that the model can generalize very well for small values of K. The best results are obtained with $K = 0.25$, being that the average $\hat{\lambda}_2 < 0.56$ for all evaluated map sizes can go as low as $\hat{\lambda}_2 = 0.52$. The second-best result is for $K = 0$, which is the same as the competitive reward structure, where the reward function only depends on the ego agent. As the K value increases, the dependence on the other agents also increases, and the model shows more difficulties in generalizing. This is notorious with $K = 0.75$, where the model struggles with larger maps but is still competent at smaller ones, and even more noticeable with $K = 1$, where it struggles in both situations.

On the other hand, the results for the NSI reward structure with the Rainbow DQN are shown in Fig. 9(b). When compared to the SI reward structure, it is clear that the results are significantly worse. All the models trained with the NSI reward structure fail to achieve generalization to all map sizes, as they all start to struggle as the maps get larger. Not only that, but for any map size, the SI counterpart achieves lower $\hat{\lambda}_2$. Interestingly, the NSI reward structure is less sensitive to the hyperparameter K as all values yield identical results. This can stem from the fact that the contribution of the ego-agent is the same for all values of K.

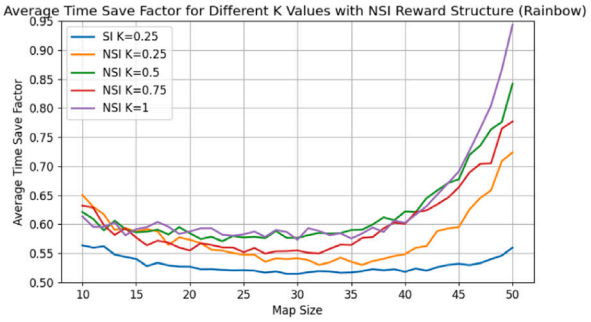
Changing the focus to the VDN, it is important to note that the hyperparameter K does not have the same impact. In the case of the SI reward, due to the reward being a unique team signal, it yields the same rewards independently of K. On the other hand, for the NSI case, K does not change cooperativeness levels, simply the importance given to visiting cells of interest(thus losing size invariance). The results are displayed in Fig. 9(c). Indeed, they are mostly analogous to the Rainbow DQN results, showing that despite transforming the individual rewards into a singular team reward, the desirable properties of the value function are still kept. In the case of the VDN, the model trained with the NSI reward performs well in smaller maps, but fails to generalize to maps larger than 30×30, i.e, the ones not seen during training. Empirically, this shows that independently of the method used, the size invariant property is desirable and important when addressing the domain shift in the form of varying map sizes.

6.3.2. Robustness

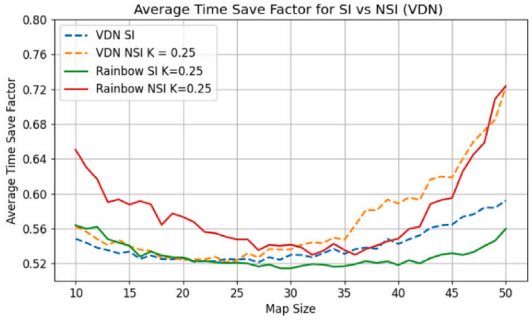
The features that have yet to be tested for effectiveness are robustness and safety filters. To evaluate how the filter changes the agents' policy, we deploy the baseline models of Rainbow DQN and VDN, but without the safety and robustness filter altering the policies. One important result is that the median time save factors for the VDN with filter is $\lambda_2 = 0.542$, whereas without it is around $\lambda = 0.540$. This shows that our action filter has a very minimal effect on the typical performance. However, looking solely at the median λ_2 does not tell the whole story. The removal of the filter during deployment significantly hinders the robustness, leading to a significant number of outliers and unfinished episodes. The effects of deploying the baseline model without the safety and robustness filter can be visualized in Fig. 10, for both the VDN and Rainbow DQN models. We consider any episode where $\lambda_2 > 0.7$ an outlier, and unfinished episodes are when the battery of all UAVs has run out, which would correspond to a failed



(a) Comparison of SI Reward Structures for 2 Agents.

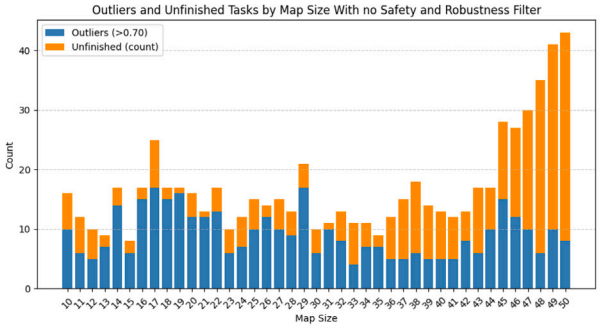


(b) Comparison of NSI Reward Structures for 2 Agents.

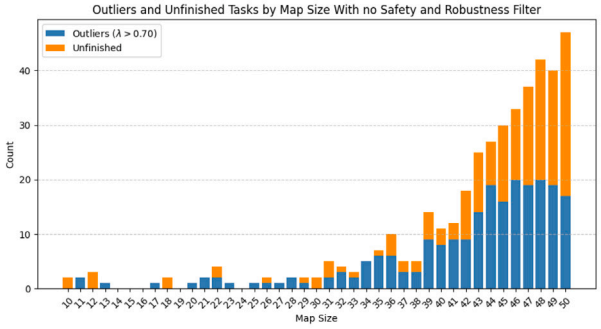


(c) Comparison of SI/NSI Reward Structures for the VDN.

Fig. 9. Comparison of size invariant and non-size invariant reward structures.



(a) Analysis of Outliers and Unfinished Tasks in the Filter Ablation Scenario, Rainbow DQN Model.



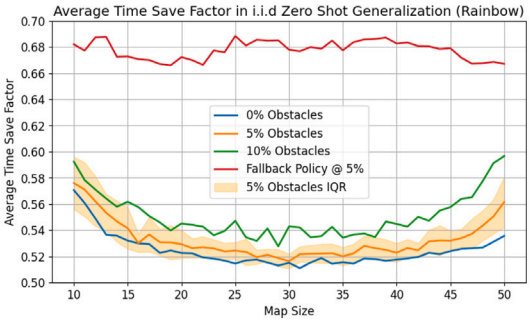
(b) Analysis of Outliers and Unfinished Tasks in the Filter Ablation Scenario, VDN Model.

Fig. 10. Comparison of ablation test results and outlier analysis for the no safety and robustness filter ablation test.

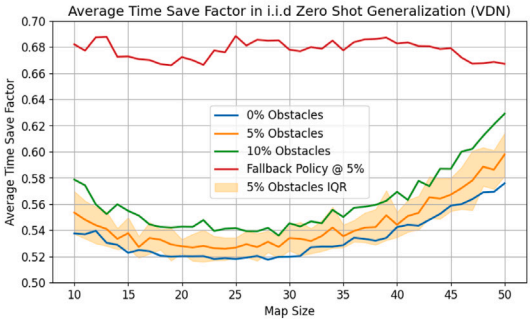
task. From the results seen in Figs. 10(a) and 10(b), one can infer that despite the Rainbow DQN having a better performance in the scenarios with two agents, it relies significantly more on the filter to achieve its results, struggling with outliers throughout all map sizes. On the other hand, the VDN method requires the action filter on maps larger than the ones seen during training, being much more robust for maps smaller than 30×30.

Focusing on the VDN results from Fig. 10(b), one can infer that 10.98% of the episodes are considered outliers, and 8.98% are unfinished. It is also important to note that 92.93% of unfinished episodes happen when the map size is larger than 30, clearly showing the challenges of the domain shift. From the results, it is evident that the safety and robustness filter is essential to guarantee the reliability of the framework, as when it is used, there are only two instances of outliers, corresponding to approximately 0.01% of all episodes, and the completion rate is 100%. Thus, while there are times when the filter does not act during the whole episode or the difference in performance is not noticeable, as seen per the median results being similar, the framework requires the filter to guarantee reliability in all scenarios, and is a valuable to tool to handle the challenges of domain shift in the form of varying map sizes.

It is worth pointing out that in all episodes, we assume that each agent has enough battery to cover the map at least 5 times, that is $b_0^j = 5|\mathbb{C}_0|$, which might be an unrealistic assumption in real-world scenarios. Nonetheless, based on the analysis of 2050 episodes for the VDN model, the median time-saving factor is $\lambda_2 = 0.54$, while the worst-case value is $\lambda_2 = 0.73$. Interpreted from a single-agent perspective, this indicates that the agents collectively consume only 54% and 73%, respectively, of the battery that a single agent would require to complete the same task. Conversely, when compared to the

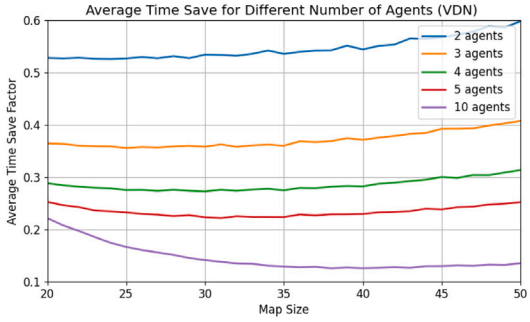


(a) Mean λ for the Monte-Carlo Simulations with $J = 2$, Rainbow DQN Model.

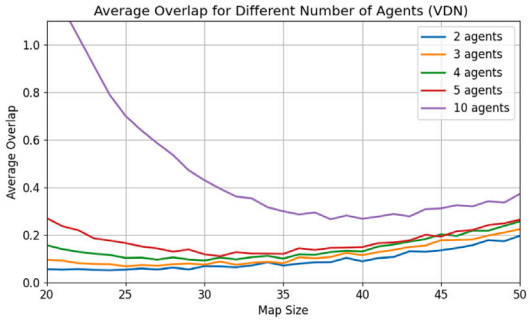


(b) Mean λ for the Monte-Carlo Simulations with $J = 2$, VDN Model.

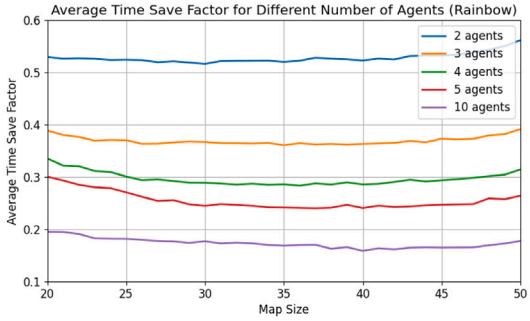
Fig. 11. Comparison of mean λ for the Monte-Carlo simulations with VDN and Rainbow.



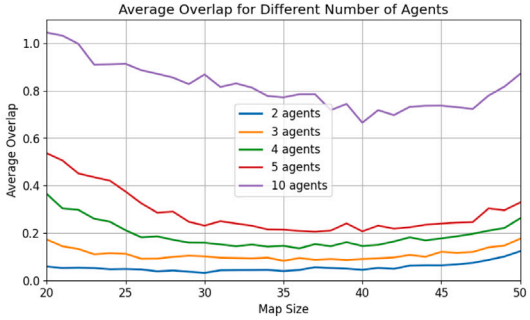
(a) Comparison of $\hat{\lambda}$ for different number of agents, VDN Model.



(b) Comparison of $\hat{O}_J$ for different number of agents, VDN Model.



(c) Comparison of $\hat{\lambda}_J$ for different number of agents, Rainbow Model



(d) Comparison of $\hat{O}_J$ for different number of agents, Rainbow Model

Fig. 12. Zero-shot generalization results for different number of agents, Rainbow DQN and VDN.

theoretical minimum battery usage achievable by two agents, these values represent an 8% and 46% increase over the optimal consumption in the median and worst-case scenarios, respectively. Since the recharge problem is not considered in this paper, we deem these battery consumption levels sufficiently low to consider that the robustness filter indeed guarantees completeness. Finally, despite being out of scope of the recharge problem, our framework is well-suited to solve it, or for practical deployments, as due to the decentralized approach, a (human) controller can deploy or remove the agents without changing the planning algorithm.

When it comes to safety, naturally, by Theorem 1, our method is safe by construction, as there are no collisions. This safety guarantee comes with some simplifications of the collision model, i.e, not considering latency and position uncertainty. Nonetheless, our algorithm works on a planning layer, so that the safety guarantees are based on the fact that the computed paths are collision-free. Moreover, in real-world implementations, one can adjust the size of the map cells in order to accommodate these uncertainties, robustifying the method.

6.3.3. Zero-shot generalization in i.i.d environments

With the ablations and comparisons done, our focus now shifts to evaluating how good the methods' zero-shot generalization (ZSG) capabilities are.

The first case considered is testing whether the VDN-based method generalizes to scenarios with two agents and maps with more or fewer obstacles than in training. To this end, on top of the 5% obstacle dataset, we also consider the 10% and obstacle-free case. We conducted Monte-Carlo simulations throughout all the datasets and compared the performance with the proposed fallback policy $\pi^{fallback}$. The results are depicted in Fig. 11.

The results clearly demonstrate the ZSG capabilities of the framework. On the 5% dataset, we have also computed the Inter Quartile Range (IQR) for the time save factor λ_2, which is typically ± 0.01 between the average $\hat{\lambda}_2$, showing the consistency of the method throughout the whole map-size spectrum. Moreover, duplicating the expected number of obstacles only slightly decreases the method's performance while still being able to generalize to all map sizes of the dataset. When removing all obstacles, as expected, the algorithm performs slightly better than the 5% baseline, showing near-optimal performance. Finally, compared with the fallback policy, the trained method typically performs better by more than 10 percentage points, showing that the method does not simply imitate the fallback policy, as it achieves a significantly better strategy for the task.

Other than generalizing to different map sizes, it is also important to evaluate the ability to generalize to different numbers of agents. Thus, our model trained with $J_{max} = 2$ is now tested in scenarios with $J \in \{2, 3, 4, 5, 10\}$ on the 5% obstacle dataset. The results can be visualized in Fig. 12 for both the VDN and Rainbow DQN. Looking first at the $\hat{\lambda}_J$ metric in Fig. 12(a) and Fig. 12(c), the models show the ability to adapt to different numbers of agents, despite never training in those situations. The curves are mainly parallel to the x-axis, demonstrating that the performance is not very sensitive to the map size. However, it is noticeable that the Rainbow DQN generalizes better to different map sizes than the VDN, as there is a slightly growing trend on the $\hat{\lambda}_2$ in the VDN as the map size increases.

Nonetheless, the performance improvements that one can get by increasing the number of agents are significant, being that for $J < 5$, $\hat{\lambda}_{J+1} < \lambda^*_J$, such that for that one can always make an argument that the number of agents can be increased to improve efficacy. However, in the case of the Rainbow DQN, the returns diminish as the number of agents increases. Not only does the optimal time save factor λ^*_J decrease with the rate $\frac{1}{J}$, but also, the method gets less efficient. This can be seen in Fig. 12(d), whereas the number of agents increases, so does the path overlap. However, this degradation in efficiency is not as noticeable for the VDN; in fact, the average Overlap $\hat{O}_J$ is very similar for all $J < 5$.

This result helps paint a more global picture of the advantages and disadvantages of both methods. On one hand, due to the architecture and focus on the single-agent task during learning, the Rainbow DQN is more efficient in generalizing to different map sizes, as in general, the agents are more capable of covering the map by themselves. Nevertheless, this focus on the ego-agent makes cooperativeness more difficult, and the generalization to a different number of agents is not as effective. In the case of the VDN, the fact that the method learns a global value function makes it easier to learn cooperative behavior. Intuitively, this yields better results in scenarios with more agents, where collaboration is necessary, but struggles slightly more in scenarios where the agents are fewer and more spread out, having an emphasis on their single-agent proficiency.

6.4. Zero-shot generalization in out-of-distribution environments

While the framework was shown to be able to generalize to different map sizes and different numbers of agents, the tests are randomly generated maps that, while different than the ones during training, still follow a very similar distribution. The most demanding setting is when maps are out-of-distribution. We will consider four maps from [76], which are not only larger than the ones seen during training but are also structured, corresponding to urban areas where the obstacles are not randomly dispersed but in clusters and forming narrow corridors. The agents' starting positions were chosen randomly, as seen in Fig. 13, and the results are available in Table 4. Note that the results are not directly comparable to the work of [76], mainly because the focus of that work was to solve instances where the UAVs had to recharge during the mission.

The results show that the VDN model can still generalize, being that the results are primarily in line with the ones obtained in the i.i.d ZSG dataset, with a couple of outliers, most notably the map Urban with two agents and three agents. Nonetheless, the results show that the model successfully adapted to structured environments that were never seen during training.

Finally, to show the versatility of the environment, we consider a simulated search-and-rescue case, where there is a person needing aid at the coordinates at a random position $c \in \mathcal{A}$. The UAVs do not have any information on where the person could be, and whenever an agent is in that exact cell, it must abandon the cover mission to give aid. As the framework works in a decentralized manner, all the other agents must continue their covering tasks as before. The results can be visualized in Fig. 14. It clearly shows that the framework is adequate for this task. Interestingly, when analyzing the mean time save factor $\hat{\lambda}_J$, it is very similar to the results obtained for $\hat{\lambda}_{J-1}$ in Table 4, which showcases the ability of the framework to cope with the removal of one agent during the coverage task. Moreover, when analyzing the average time (in steps) it took to find the missing person, increasing the number of agents J has mostly a positive effect, as there is a decreasing trend in the metric.

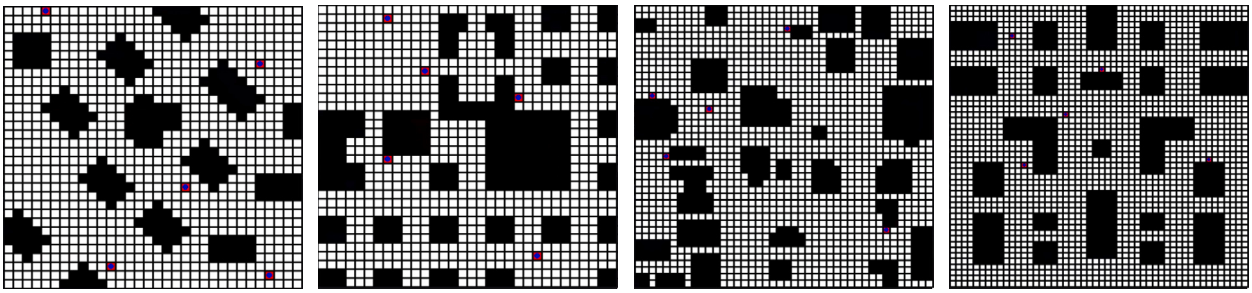
7. Conclusions

In recent years, Multi-Agent Reinforcement Learning has appeared as a valuable to unlock the potential of Multi-Robot Systems as they become more relevant in real-world applications. Despite this demand for autonomy, multi-agent approaches are still an open problem, coming with many inherent challenges, including scalability, adaptability, and robustness. All these challenges are exacerbated when considering real-world applications, where most RL and MARL solutions struggle to perform and generalize. One of the main issues is that training environments can never be as rich and dynamic as the ones agents might face in reality, so even with the development of more realistic simulators, it might not be enough to overcome the Sim2Real gap.

In this paper, we present a novel framework for Multi-Agent Coverage Path Planning (mCPP) using dynamic UAV networks. We focus on tackling this problem from a zero-shot generalization perspective, where we address the issues of lack of robustness and Sim2Real gap by developing theoretical tools that prioritize generalization over algorithm performance. To this end, we formulated the mCPP problem as an optimization problem, where a decentralized network of UAVs must minimize the time taken to cover a whole area of interest. Then, this problem is translated into a Stochastic Game, which has an adequate observation space that enables the identification of the ego agent by centering the map on its position while also compressing global information so that the representation is scalable to any map size. Moreover, we design a reward structure that enables the designer to tune the level of cooperativeness of the agents while keeping the value function size invariant, which helps with domain shift in the form of varying map sizes. Another novel feature is the design and use of a safety and robustness filter, which we show to guarantee the robustness, completeness, and safety of our method, even in worst-case scenarios.

This Stochastic Game formulation is then solved with Multi-Agent Reinforcement Learning techniques. We use a tailored version of the Rainbow DQN that is adapted to the Multi-Agent Setting by being parameter shared with a Centralized Training and Decentralized Execution (CTDE) architecture, and Value Decomposition Networks that also exploit CTDE. These methods are then combined with a novel training pipeline, with curriculum learning, domain randomization, and transfer learning, contributing to efficient learning and systematic generalization, enabling the UAV network to progressively adapt from simpler tasks to increasingly complex scenarios.

Our proposed solution demonstrates strong zero-shot generalization in scenarios that are also randomly generated but differ significantly from those encountered during training. These environments have map sizes that can be almost three times larger, have double the number of

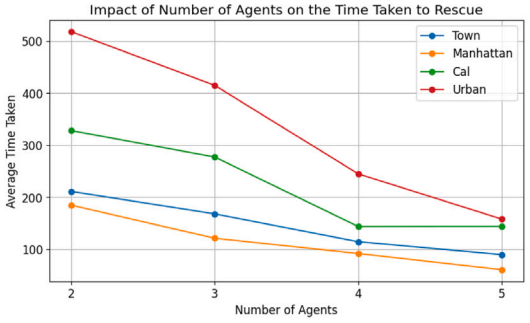


(a) Town 32×32 (b) Manhattan 32×32 (c) Cal 42×42 (d) Urban 50×50

Fig. 13. The four maps used for the out-of-distribution zero-shot generalization dataset [76].



(a) Comparison of average $\hat{\lambda}$ for different number of agents.



(b) Comparison of average time (in steps) that it takes to rescue the person.

Fig. 14. Zero-shot generalization results for the search-and-rescue task.

Table 4
Time save factor λ results for out-of-distribution zero-shot generalization dataset.

Environment	2 Agents	3 Agents	4 Agents	5 Agents
Town 32×32	0.56	0.38	0.31	0.25
Manhattan 32×32	0.53	0.36	0.30	0.25
Cal 42×42	0.58	0.39	0.32	0.30
Urban 50×50	0.64	0.42	0.36	0.29

obstacles, and have up to five times more agents than what was seen during training. In these scenarios, we show that our method can still achieve near-optimal solutions while still being very robust, achieving 100% episode completion rate across more than 14000 scenarios. This ability to generalize without retraining makes the framework highly suitable for real-world applications that are unpredictable and dynamic and where rapid deployment is essential, such as disaster relief and search-and-rescue missions.

In addition to these scenarios, which are still randomly generated, the framework's adaptability extends to handling tasks in structured map environments that are clearly outside of the training distribution. Across four structured maps [76], with a very large number of clustered obstacles and narrow corridors, we show that our method can generalize while still keeping comparable performance without additional training. Moreover, in a search-and-rescue setting, the method was able to cope with the removal of one agent from the network while still successfully and efficiently covering the map.

Furthermore, we also provided an in-depth discussion on the trade-offs between using adapted single-agent methods such as the Rainbow DQN and SOTA MARL methods like the VDN. We empirically demonstrate that the Rainbow DQN deals better with domain shift caused by different map sizes, whereas the VDN is better suited to adapt to different numbers of agents during deployment. We theorize that these differences are inherent to the methods learning framework, where one learns from a team reward signal, prioritizing cooperation, and the other learns from an individual reward (with a shared term), which improves the path efficiency from the single-agent point of view. These advancements establish a robust and generalizable solution for mCPP in dynamic environments, laying the groundwork for further research and practical implementation. Future research can include the study of more powerful MARL methods that combine the best aspects from the Rainbow DQN and the VDN. This could include adapting the VDN method to allow different numbers of agents during training. Another interesting avenue for further research is the usage of more advanced neural network architectures, like graph neural networks (GNN), Long Short-Term Memory (LSTM), and Transformers. Other directions can include the extension of the framework to heterogeneous multi-agent systems and exploring more demanding scenarios where the battery levels are more constraining, requiring recharging during the mission. Moreover, formally addressing challenges such as limited communication and more complex dynamic interactions in real-world scenarios will further enhance the applicability of this type of approach.

CRediT authorship contribution statement

José P. Carvalho: Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis. **A. Pedro Aguiar:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was conducted under the FCT doctoral fellowship with the reference number 2024.01894.BD, and was partially supported by the project F2ForAll (NORTE2030-FEDER-02700700), and R&D Unit SYSTEC - UID/00147 funded by FCT/MECI through national funds funds.

Data availability

Data will be made available on request.

References

[1] J. Cortés, M. Egerstedt, Coordinated control of multi-robot systems: A survey, SICE J. Control. Meas. Syst. Integr. 10 (6) (2017) 495–503.
[2] T. Arai, E. Pagello, L.E. Parker, et al., Advances in multi-robot systems, IEEE Trans. Robot. Autom. 18 (5) (2002) 655–661.
[3] P. Radoglou-Grammatikis, P. Sarigiannidis, T. Lagkas, I. Moscholios, A compilation of UAV applications for precision agriculture, Comput. Netw. 172 (2020) 107148.
[4] M. Ramasamy, D. Ghose, Learning-based preferential surveillance algorithm for persistent surveillance by unmanned aerial vehicles, in: 2016 International Conference on Unmanned Aircraft Systems, ICUAS, IEEE, 2016, pp. 1032–1040.
[5] C. Di Franco, G. Buttazzo, Coverage path planning for UAVs photogrammetry with energy and resolution constraints, J. Intell. Robot. Syst. 83 (2016) 445–462.
[6] H. Zhang, X. Zhang, Z. Feng, X. Xiao, Heterogeneous multi-robot cooperation with asynchronous multi-agent reinforcement learning, IEEE Robot. Autom. Lett. 9 (1) (2024) 159–166.
[7] R.M. Claro, M.I. Pereira, F.S. Neves, A.M. Pinto, Energy efficient path planning for 3D aerial inspections, IEEE Access 11 (2023) 32152–32166.
[8] E. Seraj, M. Gombolay, Coordinated control of uavs for human-centered active sensing of wildfires, in: 2020 American Control Conference, ACC, IEEE, 2020, pp. 1845–1852.
[9] Z. Xiaoning, Analysis of military application of UAV swarm technology, in: 2020 3rd International Conference on Unmanned Systems, ICUS, IEEE, 2020, pp. 1200–1204.
[10] S.M. Ayanz, et al., Forest Fires in Europe, Middle East and North Africa 2021, (no. KJ-NA-31-269-EN-N (online),KJ-NA-31-269-EN-C (print)) Publications Office of the European Union, Luxembourg (Luxembourg), 2022.
[11] H. Choset, Coverage for robotics - A survey of recent results, Ann. Math. Artif. Intell. 31 (2001) 113–126.
[12] R. Menezes, F. Martins, F.E. Vieira, R. Silva, M. Braga, A model for terrain coverage inspired by ant's alarm pheromones, in: Proceedings of the 2007 ACM Symposium on Applied Computing, SAC '07, Association for Computing Machinery, New York, NY, USA, 2007, pp. 728–732.
[13] K. Easton, J. Burdick, A coverage algorithm for multi-robot boundary inspection, in: Proceedings of the 2005 IEEE International Conference on Robotics and Automation, 2005, pp. 727–734.
[14] I. Rekleitis, A.P. New, E.S. Rankin, H. Choset, Efficient boustrophedon multi-robot coverage: an algorithmic approach, Ann. Math. Artif. Intell. 52 (2008) 109–142.
[15] C. Berner, et al., Dota 2 with large scale deep reinforcement learning, 2019, arXiv preprint arXiv:1912.06680.
[16] J. Westheider, J. Rückin, M. Popović, Multi-UAV adaptive path planning using deep reinforcement learning, in: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2023, pp. 649–656.
[17] S.Y. Luis, D.G. Reina, S.L.T. Marín, A multiagent deep reinforcement learning approach for path planning in autonomous surface vehicles: The Ypacaraí Lake Patrolling Case, IEEE Access 9 (2021) 17084–17099.
[18] M. Jana, L. Vachhani, A. Sinha, A deep reinforcement learning approach for multi-agent mobile robot patrolling, Int. J. Intell. Robot. Appl. 6 (4) (2022) 724–745.
[19] J.K. Terry, N. Grammel, S. Son, B. Black, A. Agrawal, Revisiting parameter sharing in multi-agent deep reinforcement learning, 2023.
[20] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, I. Mordatch, Multi-agent actor-critic for mixed cooperative-competitive environments, 2020.
[21] C. Amato, An introduction to centralized training for decentralized execution in cooperative multi-agent reinforcement learning, 2024.
[22] G. Papoudakis, F. Christianos, A. Rahman, S.V. Albrecht, Dealing with non-stationarity in multi-agent deep reinforcement learning, 2019.
[23] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, D. Silver, Rainbow: combining improvements in deep reinforcement learning, in: Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence, in: AAAI'18/IAAI'18/EAAI'18, AAAI Press, 2018.
[24] K.-C. Hsu, H. Hu, J.F. Fisac, The safety filter: A unified view of safety-critical control in autonomous systems, Annu. Rev. Control. Robot. Auton. Syst. 7 (Volume 7, 2024) (2024) 47–72.
[25] E. Galceran, M. Carreras, A survey on coverage path planning for robotics, Robot. Auton. Syst. 61 (2013) 1258–1276.
[26] I.A. Wagner, M. Lindenbaum, A.M. Bruckstein, Efficiently searching a graph by a smell-oriented vertex process, Ann. Math. Artif. Intell. 24 (1) (1998) 211–223.
[27] D. Kurabayashi, J. Ota, T. Arai, E. Yoshida, Cooperative sweeping by multiple mobile robots, in: Proceedings of IEEE International Conference on Robotics and Automation, vol. 2, 1996, pp. 1744–1749.
[28] T.M. Cabreira, L.B. Brisolara, P.R. Ferreira Jr., Survey on coverage path planning with unmanned aerial vehicles, Drones 3 (1) (2019).
[29] C.S. Tan, R. Mohd-Mokhtar, M.R. Arshad, A comprehensive review of coverage path planning in robotics using classical and heuristic algorithms, IEEE Access 9 (2021) 119310–119342.
[30] A. Mannan, M.S. Obaidat, K. Mahmood, A. Ahmad, R. Ahmad, Classical versus reinforcement learning algorithms for unmanned aerial vehicle network communication and coverage path planning: A systematic literature review, Int. J. Commun. Syst. 36 (5) (2023).
[31] C.J. Watkins, P. Dayan, Technical note: Q-learning, Mach. Learn. 8 (1992) 279–292.
[32] R.S. Sutton, A.G. Barto, Reinforcement learning: An introduction, MIT Press, 2018.
[33] A. Nezamzadeh, H.J. Bidgoly, M. Kamali, Cooperative coverage path planning using Q-learning and sarsa in two environments, in: 2024 32nd International Conference on Electrical Engineering, ICEE, 2024, pp. 1–5.
[34] J. Xiao, G. Wang, Y. Zhang, L. Cheng, A distributed multi-Agent Dynamic Area coverage algorithm based on reinforcement learning, IEEE Access 8 (2020) 33511–33521.
[35] A. Guerra, F. Guidi, D. Dardari, P.M. Djurić, Reinforcement learning for joint detection and mapping using dynamic UAV networks, IEEE Trans. Aerosp. Electron. Syst. 60 (3) (2024) 2586–2601.
[36] H. Bayerlein, M. Theile, M. Caccamo, D. Gesbert, UAV path planning for wireless data harvesting: A deep reinforcement learning approach, in: GLOBECOM 2020 - 2020 IEEE Global Communications Conference, 2020, pp. 1–6.
[37] M. Theile, H. Bayerlein, R. Nai, D. Gesbert, M. Caccamo, UAV coverage path planning under varying power constraints using deep reinforcement learning, in: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2020, pp. 1444–1449.
[38] M. Theile, H. Bayerlein, R. Nai, D. Gesbert, M. Caccamo, UAV path planning using global and local map information with deep reinforcement learning, in: 2021 20th International Conference on Advanced Robotics, ICAR 2021, Institute of Electrical and Electronics Engineers Inc., 2021, pp. 539–546.
[39] H. Bayerlein, M. Theile, M. Caccamo, D. Gesbert, Multi-UAV path planning for wireless data harvesting with deep reinforcement learning, IEEE Open J. Commun. Soc. 2 (2021) 1171–1187.
[40] Z. Dong, C. Liu, Collaborative coverage path planning of UAV cluster based on deep reinforcement learning, in: 2021 IEEE 3rd International Conference on Frontiers Technology of Information and Computer, ICFTIC, 2021, pp. 201–207.
[41] A. Din, M.Y. Ismail, B. Shah, M. Babar, F. Ali, S.U. Baig, A deep reinforcement learning-based multi-agent area coverage control for smart agriculture, Comput. Electr. Eng. 101 (2022) 108089.
[42] F. Venturini, F. Mason, F. Pase, F. Chiariotti, A. Testolin, A. Zanella, M. Zorzi, Distributed reinforcement learning for flexible UAV swarm control with transfer learning capabilities, in: Proceedings of the 6th ACM Workshop on Micro Aerial Vehicle Networks, Systems, and Applications, DroNet '20, Association for Computing Machinery, New York, NY, USA, 2020.
[43] D.S. Diop, S.Y. Luis, M.P. Esteve, S.L.T. Marín, D.G. Reina, Decoupling patrolling tasks for water quality monitoring: A multi-agent deep reinforcement learning approach, IEEE Access 12 (2024) 75559–75576.
[44] A.M. Barrionuevo, S. Yanes Luis, D. Gutiérrez Reina, S.L. Toral Marín, Informative deep reinforcement path planning for heterogeneous autonomous surface vehicles in large water resources, IEEE Access 12 (2024) 71835–71852.

[45] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017.
[46] J. Bialas, M. Doeller, R. Kathrein, Robust multi-agent coverage path planning for unmanned airial vehicles (UAVs) in complex 3D environments with deep reinforcement learning*, in: 2023 IEEE International Conference on Robotics and Biomimetics, ROBIO, 2023, pp. 1–6.
[47] M. Xu, Y. She, Y. Jin, J. Wang, Deep reinforcement learning for image-based multi-agent coverage path planning, in: 2023 IEEE 98th Vehicular Technology Conference, VTC2023-Fall, 2023, pp. 1–7.
[48] Y. Hu, J. Fu, G. Wen, Graph soft actor–critic reinforcement learning for large-scale distributed multirobot coordination, IEEE Trans. Neural Netw. Learn. Syst. 36 (1) (2025) 665–676.
[49] Y. Li, A.H. Aghvami, D. Dong, Path planning for cellular-connected UAV: A DRL solution with quantum-inspired experience replay, IEEE Trans. Wirel. Commun. 21 (10) (2022) 7897–7912.
[50] R. Kirk, A. Zhang, E. Grefenstette, T. Rocktäschel, A survey of zero-shot generalisation in deep reinforcement learning, J. Artif. Int. Res. 76 (2023).
[51] F.A. Oliehoek, Decentralized POMDPs, in: Reinforcement Learning: State-of-the-Art, Springer Berlin Heidelberg, Berlin, Heidelberg, 2012, pp. 471–503.
[52] D.S. Bernstein, R. Givan, N. Immerman, S. Zilberstein, The Complexity of Decentralized Control of Markov Decision Processes, vol. 27, INFORMS, 2002, pp. 819–840, http://dx.doi.org/10.1287/MOOR.27.4.819.297.
[53] J.P. Carvalho, A.P. Aguiar, Deep reinforcement learning for zero-shot coverage path planning with mobile robots, IEEE/CAA J. Autom. Sin. 12 (8) (2025) 1594–1609.
[54] F. Pardo, A. Tavakoli, V. Levdik, P. Kormushev, Time limits in reinforcement learning, 2022.
[55] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing atari with deep reinforcement learning, 2013.
[56] V. Mnih, et al., Human-level control through deep reinforcement learning, Nat. 2015 518:7540 518 (2015) 529–533.
[57] Y. Efroni, C. Jin, A. Krishnamurthy, S. Miryoosefi, Provable reinforcement learning with a short-term memory, in: International Conference on Machine Learning, PMLR, 2022, pp. 5832–5850.
[58] K. Zhang, Z. Yang, T. Başar, Multi-agent reinforcement learning: A selective overview of theories and algorithms, Stud. Syst. Decis. Control. 325 (2019) 321–384.
[59] J.V. Messias, M.T. Spaan, P.U. Lima, Multiagent POMDPs with asynchronous execution, in: Proceedings of the 2013 International Conference on Autonomous Agents and Multi-Agent Systems, 2013, pp. 1273–1274.
[60] R. Becker, S. Zilberstein, V. Lesser, Decentralized Markov decision processes with event-driven interactions, in: Proceedings of the Third International Joint Conference on Autonomous Agents and Multiagent Systems, 2004, AAMAS 2004, 2004, pp. 302–309.
[61] Y. Cai, C. Daskalakis, On minmax theorems for multiplayer games, in: Proceedings of the Twenty-Second Annual ACM-SIAM Symposium on Discrete Algorithms, SODA '11, Society for Industrial and Applied Mathematics, USA, 2011, pp. 217–234.
[62] A. Kanervisto, C. Scheller, V. Hautamäki, Action space shaping in deep reinforcement learning, in: 2020 IEEE Conference on Games, CoG, 2020, pp. 479–486.
[63] T. Lam, A. Verma, B.K.H. Low, P. Jaillet, Risk-aware reinforcement learning with coherent risk measures and non-linear function approximation, in: The Eleventh International Conference on Learning Representations, 2022.
[64] P. Sunehag, G. Lever, A. Gruslys, W.M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J.Z. Leibo, K. Tuyls, T. Graepel, Value-decomposition networks for cooperative multi-agent learning based on team reward, in: Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems, AAMAS '18, Richland, SC, 2018, pp. 2085–2087.
[65] C. Claus, C. Boutilier, The dynamics of reinforcement learning in cooperative multiagent systems, in: Proceedings of the Fifteenth National/Tenth Conference on Artificial Intelligence/Innovative Applications of Artificial Intelligence, in: AAAI '98/IAAI '98, American Association for Artificial Intelligence, USA, 1998, pp. 746–752.
[66] S. Sen, M. Sekaran, J. Hale, Learning to coordinate without sharing information, in: Proceedings of the Twelfth AAAI National Conference on Artificial Intelligence, AAAI '94, AAAI Press, 1994, pp. 426–431.
[67] S.V. Albrecht, F. Christianos, L. Schäfer, Multi-Agent Reinforcement Learning: Foundations and Modern Approaches, MIT Press, 2024.
[68] E. Rodrigues Gomes, R. Kowalczyk, Dynamic analysis of multiagent Q-learning with ϵ-greedy exploration, ICML '09, Association for Computing Machinery, New York, NY, USA, 2009, pp. 369–376.
[69] J.K. Gupta, M. Egorov, M. Kochenderfer, Cooperative multi-agent control using deep reinforcement learning, in: Autonomous Agents and Multiagent Systems: AAMAS 2017 Workshops, Best Papers, São Paulo, Brazil, May 8–12, 2017, Revised Selected Papers 16, Springer, 2017, pp. 66–83.
[70] T. Rashid, M. Samvelyan, C.S. de Witt, G. Farquhar, J.N. Foerster, S. Whiteson, QMIX: monotonic value function factorisation for deep multi-agent reinforcement learning, 2018, CoRR abs/1803.11485.
[71] M.G. Bellemare, W. Dabney, R. Munos, A distributional perspective on reinforcement learning, 2017.
[72] D. Schmidt, T. Schmied, Fast and data-efficient training of rainbow: an experimental study on atari, 2021.
[73] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, 2019.
[74] M. Fortunato, et al., Noisy networks for exploration, 2019.
[75] A. Bou, M. Bettini, S. Dittert, V. Kumar, S. Sodhani, X. Yang, G.D. Fabritiis, V. Moens, TorchRL: A data-driven decision-making library for PyTorch, 2023.
[76] M. Theile, H. Bayerlein, M. Caccamo, A.L. Sangiovanni-Vincentelli, Learning to recharge: UAV coverage path planning through deep reinforcement learning, 2023.



José Pedro Carvalho received the M.Sc. degree in Electrical and Computer Engineering (ECE) from the Faculty of Engineering, University of Porto (FEUP) in 2023. During his M.Sc. studies, he also visited the University of California, Los Angeles (UCLA), USA. He is currently pursuing the Ph.D. degree in ECE at FEUP, supported by a Ph.D. scholarship from the Portuguese Foundation for Science and Technology (FCT) in collaboration with the Royal Institute of Technology (KTH), Sweden. His research interests include reinforcement learning and learning-based control for robotics, with a particular focus on certifiably safe motion control of mobile robots.



António Pedro Aguiar received the Ph.D. in Electrical and Computer Engineering (ECE) from the Instituto Superior T cnico (IST), University of Lisbon, Portugal in 2002. He is currently a Full Professor with the ECE Department at the Faculty of Engineering, University of Porto (FEUP) and Director of ARISE - Advanced Production and Intelligent Systems Associate Laboratory. From 2002 to 2005, he was a post-doctoral researcher at the Center for Control, Dynamical-Systems, and Computation at the University of California, Santa Barbara (UCSB). From 2005 to 2012, he was a senior researcher with the Institute for Systems and Robotics at IST, and an invited professor with the ECE Department at IST, Lisbon. His research interests include modeling, control, navigation, and guidance of autonomous robotic vehicles, nonlinear control, switched and hybrid systems, tracking, path-following, performance limitations, nonlinear observers, the integration of machine vision with feedback control, networked control, and coordinated/cooperative control of multiple autonomous robotic vehicles.